Part 2 of EE6221 – Scope:

- **Introduction and Sensors** (concepts, no math)
- **Vision Sensors and Systems** (concepts, geometry, methods, math)
- **Structure and Pose Estimation** (concepts, geometry, algorithms, math)
- **Kalman Filter and Sensor Integration** (concepts, methods, math)
- **Vision-Based Control & Estimation** (concepts, methods, math)

Lec 1 - Sensors

- Introduction
- Sensor classification
- Sensor characterization
- Motor encoders (position)
- Heading/orientation sensors
- Ground-based beacons
- Active ranging
- Speed sensors
- Vision (sensors, depth from focus, stereo vision, optic flow, etc.)

Reference:

- *Introduction to Autonomous Mobile Robots (ch.4),* R. Siegwart, and I. Nourbakhsh, MIT Press, 2004
- *Sensors for Mobile Robots, Theory and Applications,* H.R. Everett, A K Peters Ltd., 1995

Internal State Sensors

- **Internal state sensors** consists of devices used to measure position, velocity, or acceleration of robot joints and/or the end effectors. The following devices are some of the sensors that belong to this class:

  - Potentiometers: measure the electrical potentials

  - Optical encoders: motion sensing device
  - Tachometers: measures the rotation speed of a shaft or disk

  - …

External State Sensors

Instruments of external state sensors (not a full list):

Vision (camera, infrared, laser, RFID) for landmark recognition, 3D scene recovery, scene understanding, etc.

Tactile sensors: Four measurable features—force, contact time, repetition, and contact area change—can effectively categorize touch patterns.

Range finders (lasers, ultrasonic) for 3D scene recovery, local position.

GPS (differential GPS) for global positioning

Odometry (encoders, gyroscopes, accelerometers) – absolute motion recovery.

The Role of Multi-Sensor Integration and Fusion

The purpose of external sensors is to provide a system with useful information concerning some features of interest in the system's environment. This enables the machine to ***act intelligently*** via some intelligent programme.

The potential advantages in ***integrating*** and/or ***fusing information*** from ***multiple sensors*** are that the information can be decided more accurately, concerning features that are impossible to perceive with individual sensors, as well as in less time, and at a lesser cost.

These advantages correspond, respectively, to the notions of the ***redundancy***, ***complementarity***, ***timeliness***, *and* ***cost*** of the information provided the system.

The Role of Multi-Sensor Integration and Fusion

Redundant information is provided from a group of sensors (or a single sensor over time) when each sensor is perceiving, possibly with a different fidelity, the same features in the environment.

The integration or fusion of redundant information ***can reduce overall uncertainty*** and thus serve to increase the accuracy with which the features are perceived by the system.

Multiple sensors providing redundant information can also serve to ***increase reliability*** in the case of sensor error or failure.

The Role of Multi-Sensor Integration and Fusion

Complementary information from multiple sensors allows features in the environment to be perceived that are impossible to perceive using just the information from each individual sensor operating separately.

More timely information, as compared to the speed at which it could be provided by a single sensor, may be provided by multiple sensors either because of the actual speed of operation of each sensor, or because of the ***processing parallelism*** that may possibly be achieved as part of tile integration process.

The Role of Multi-Sensor Integration and Fusion

Less costly information, in the context of a system with multiple sensors, is information obtained at a lesser cost as compared to the equivalent quality of information that could be obtained from a single sensor.

Unless the information provided by the single sensor is being used for additional functions in the system, the total cost of the single sensor should be compared to the total cost of the integrated multi-sensor system.

With redundancy, it is possible then to ***use lower cost sensors with lower quality*** that can be combined to achieve the same quality of reading from a high quality sensor (thereby being costly).

Classification of Sensors for Robots

Classification based on where energy comes from:

- Active sensors
  - emit their own energy and measure the reaction
  - better performance, but some influence on the environment and generally consumes more energy
  - Examples: ultrasonic, laser, and IR sensors.

- Passive sensors
  - receives energy already in the environment
  - consume less energy, but often have signal and noise problems
  - Examples: camera

General Classification

General classification (typical use)	Sensor Sensor System	PC or EC	A or P
Tactile sensors (detection of physical contact or closeness; security switches)	Contact switches, bumpers	EC	P
	Optical barriers	EC	A
	Noncontact proximity sensors	EC	A
Wheel/motor sensors (wheel/motor speed and position)	Brush encoders	PC	P
	Potentiometers	PC	P
	Synchros, resolvers	PC	A
	Optical encoders	PC	A
	Magnetic encoders	PC	A
	Inductive encoders	PC	A
	Capacitive encoders	PC	A
Heading sensors (orientation of the robot in relation to a fixed reference frame)	Compass	EC	P
	Gyroscopes	PC	P
	Inclinometers	EC	A/P

A, active; P, passive; P/A, passive/active; PC, proprioceptive; EC, exteroceptive.

(adapted from Siegwart & Nourbakhsh, 2004, ch. 4)

General Classification

General classification (typical use)	Sensor Sensor System	PC or EC	A or P
Ground-based beacons (localization in a fixed reference frame)	GPS	EC	A
	Active optical or RF beacons	EC	A
	Active ultrasonic beacons	EC	A
	Reflective beacons	EC	A
Active ranging (reflectivity, time-of-flight, and geometric triangulation)	Reflectivity sensors	EC	A
	Ultrasonic sensor	EC	A
	Laser rangefinder	EC	A
	Optical triangulation (1D)	EC	A
	Structured light (2D)	EC	A
Motion/speed sensors (speed relative to fixed or moving objects)	Doppler radar	EC	A
	Doppler sound	EC	A
Vision-based sensors (visual ranging, whole-image analysis, segmentation, object recognition)	CCD/CMOS camera(s)	EC	P
	Visual ranging packages		
	Object tracking packages		

A, active; P, passive; P/A, passive/active; PC, proprioceptive; EC, exteroceptive.

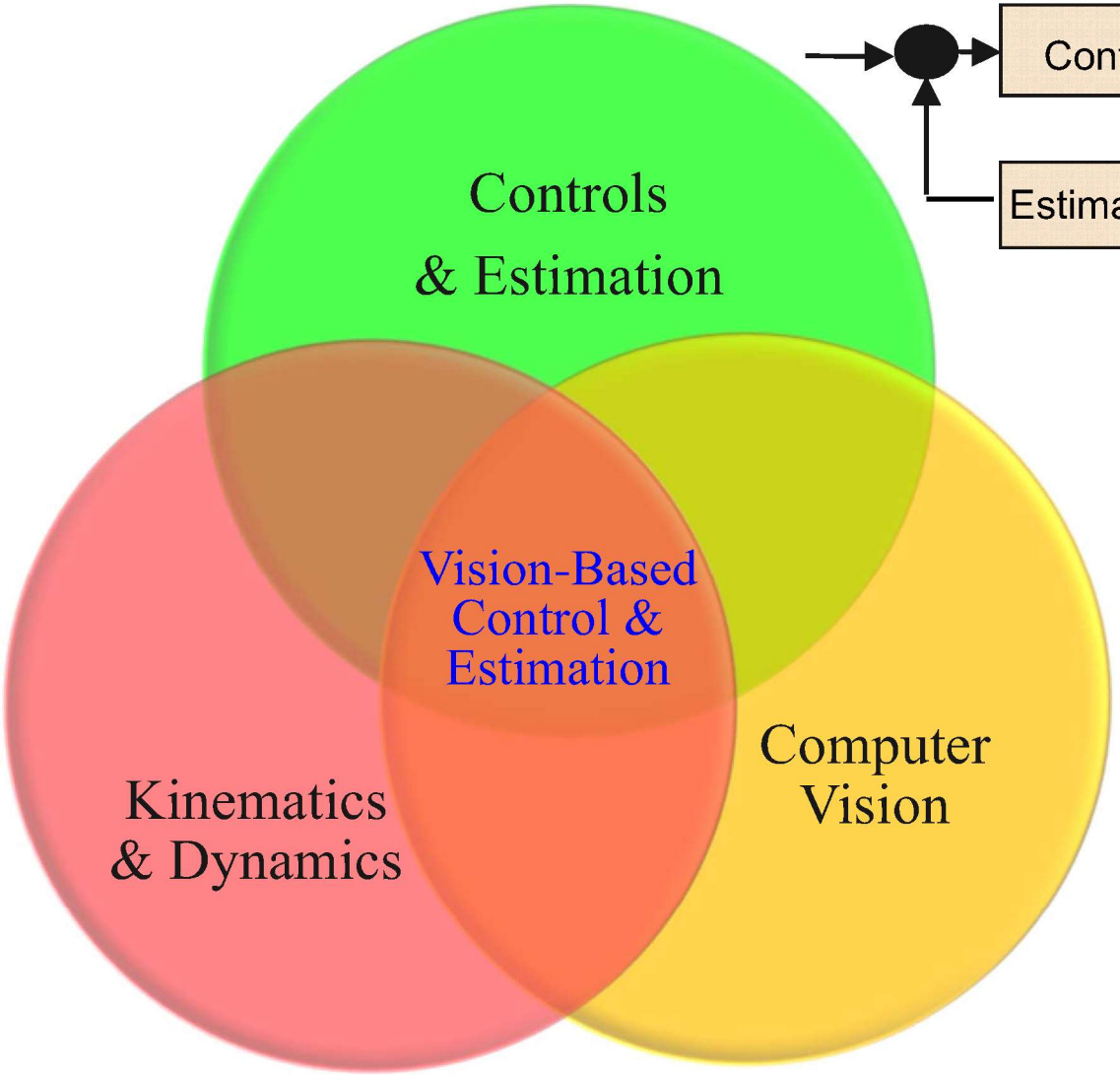
(adapted from Siegwart & Nourbakhsh, 2004, ch. 4)

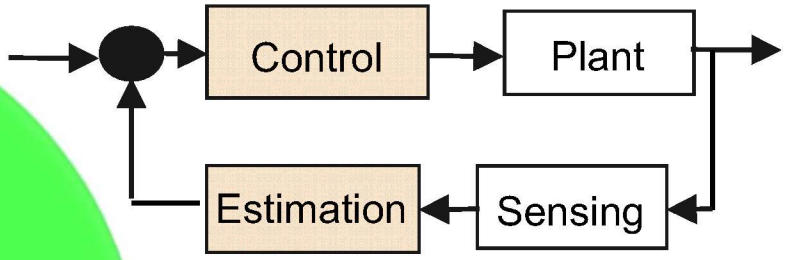
Lecture 2: Vision Sensors and Systems

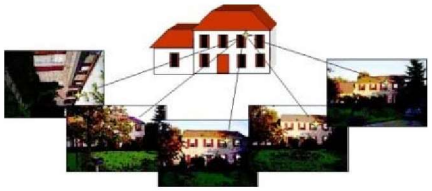
- Cameras
- Imaging
- Vision-based control & estimation
- Pinhole camera model
- Camera calibration

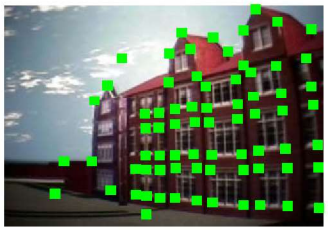
Vision-Based Control & Estimation

- Vision-based control and estimation: use of image data in feedback control and state estimation of moving agents (e.g., robot manipulators, mobile robots, or unmanned vehicles, etc).

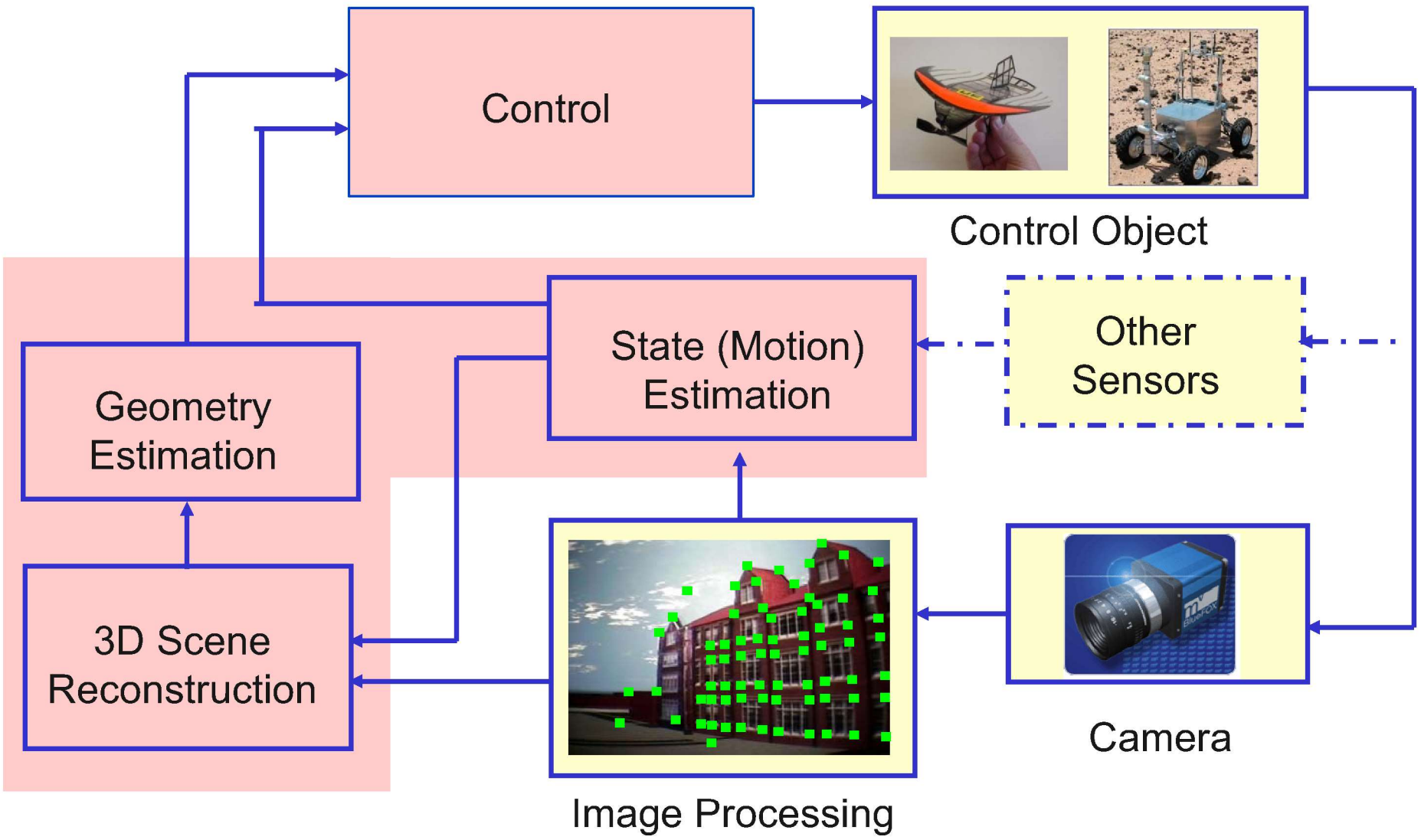








Vision-Based Control & Estimation



Digital Imaging

32x32 pixel image
Each pixel is eight bits

[0,0]

255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	250	254	233	182
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	237	207	171	192
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	244	254	178	162
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	250	253	243	252
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	253	234	231	234	213
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	253	232	221	243	250
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	252	183	181	163
255	255	255	255	255	255	255	255	255	255	254	255	255	254	254	255	255	254	254	254	254	255	255	255	255	255	255	254	252	168	181	174
255	255	255	255	255	255	255	255	255	238	222	225	214	206	200	195	183	201	202	187	197	202	203	230	253	254	255	255	237	254	161	224
255	255	255	255	255	255	255	255	255	242	202	216	214	214	216	200	197	215	215	217	205	189	197	228	255	255	255	255	246	253	153	229
255	255	255	255	255	255	255	254	255	224	201	210	206	207	224	198	207	202	193	179	206	194	189	205	254	255	255	255	252	250	163	240
253	253	251	254	254	254	255	255	251	190	210	199	220	190	200	203	212	201	223	204	184	215	202	185	246	255	252	254	250	254	156	220
254	254	255	220	183	211	215	253	189	206	214	202	220	220	202	214	211	193	191	220	212	184	182	222	209	215	193	210	118	129	158	241
255	253	254	252	181	152	160	163	171	136	125	127	126	139	113	124	117	108	111	119	132	88	141	107	127	137	136	129	107	237	159	245
163	210	247	254	238	143	130	131	119	139	140	117	121	129	127	120	124	122	105	127	128	104	127	128	84	96	89	113	253	215	150	212
157	168	160	182	185	243	130	115	111	108	103	130	118	103	101	91	96	92	94	94	94	116	101	94	85	96	96	201	255	251	151	238
95	133	126	139	187	254	237	145	138	146	130	133	153	144	147	139	146	138	153	138	135	143	128	137	128	119	225	248	251	221	137	201
107	140	119	160	134	152	169	131	133	132	117	132	127	123	119	124	118	121	120	116	116	118	108	113	122	115	135	128	105	138	123	131
85	88	104	113	127	109	185	147	109	119	139	115	138	146	128	135	131	130	137	124	129	141	136	89	109	117	82	101	70	76	88	174
99	106	94	101	109	138	206	88	75	85	85	77	86	82	82	85	77	90	69	81	79	78	73	79	73	97	139	119	95	84	147	207
97	78	116	91	101	99	200	175	102	138	130	81	82	93	87	71	94	92	128	81	75	120	123	130	77	144	188	195	139	124	88	105
98	96	123	100	93	110	190	207	127	187	179	76	90	100	91	87	102	102	160	100	91	163	177	183	97	133	220	226	119	114	115	112
103	63	70	68	111	146	207	139	141	232	237	77	98	105	96	97	110	99	189	109	92	217	215	226	101	128	253	153	82	104	153	130
60	78	99	71	70	93	233	236	148	147	125	185	180	201	203	208	208	193	198	185	190	189	135	138	135	147	138	133	126	125	106	109
57	114	50	64	89	95	133	128	155	67	242	183	183	191	202	188	198	193	209	193	195	194	193	250	159	158	173	142	159	143	134	134
63	91	70	75	86	88	133	94	109	114	104	119	130	121	130	125	146	117	131	134	126	134	105	118	123	125	97	89	127	121	113	134
45	87	109	108	116	123	67	102	100	115	117	126	104	116	109	91	90	84	113	113	102	96	122	92	97	99	115	123	139	86	118	139
59	64	103	81	119	72	56	71	84	74	64	79	72	70	86	82	116	108	67	71	71	83	73	148	117	78	86	90	86	100	90	110
81	61	93	104	91	109	120	131	129	146	153	156	163	187	187	198	189	195	201	199	187	190	206	85	71	87	75	103	77	72	39	57
139	159	153	162	160	175	174	171	181	174	173	183	177	193	182	161	184	180	175	188	176	183	183	187	165	207	212	210	194	192	187	195
157	169	170	177	187	184	178	187	172	165	171	181	202	198	202	181	201	183	189	195	174	169	172	177	163	185	170	180	203	211	205	212
173	180	177	168	165	170	166	168	170	188	185	168	193	200	187	176	171	194	203	190	185	186	165	202	192	197	175	202	184	196	193	210

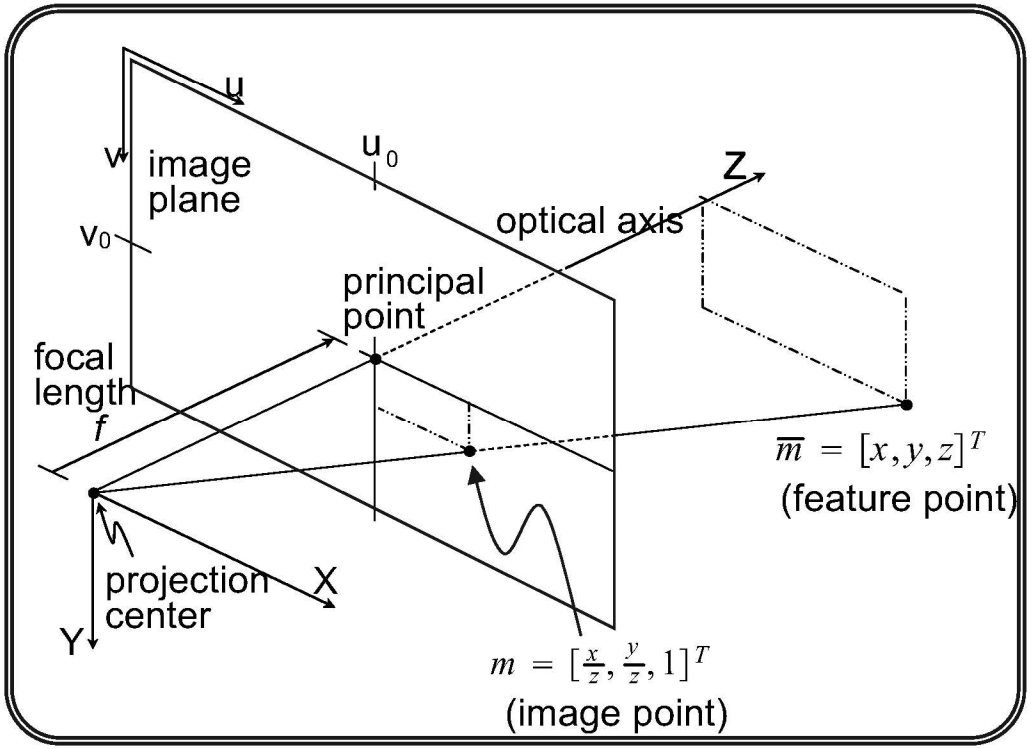
[31,31]

- For an eight-bit image, pixels take integer values in range 0…255
- Typically pixel coordinates are counted from top left to bottom right, though some cameras count from bottom left

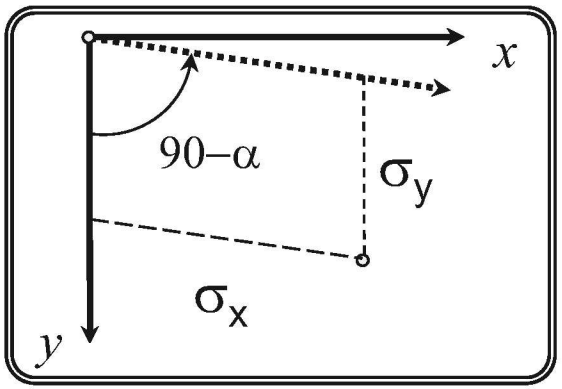
Pinhole Camera Model

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f\sigma_x & -f\sigma_x \tan\alpha & u_0 \\ 0 & f\sigma_y \sec\alpha & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{x}{z} \\ \frac{y}{z} \\ 1 \end{bmatrix}$$

$$p = Am$$



- A is intrinsic calibration matrix
- f is camera focal length
- σ_x, σ_y are size of pixels in focal lengths
- $[u_0, v_0]$ is the principal point in pixels
- α is skew angle, usually ≈0



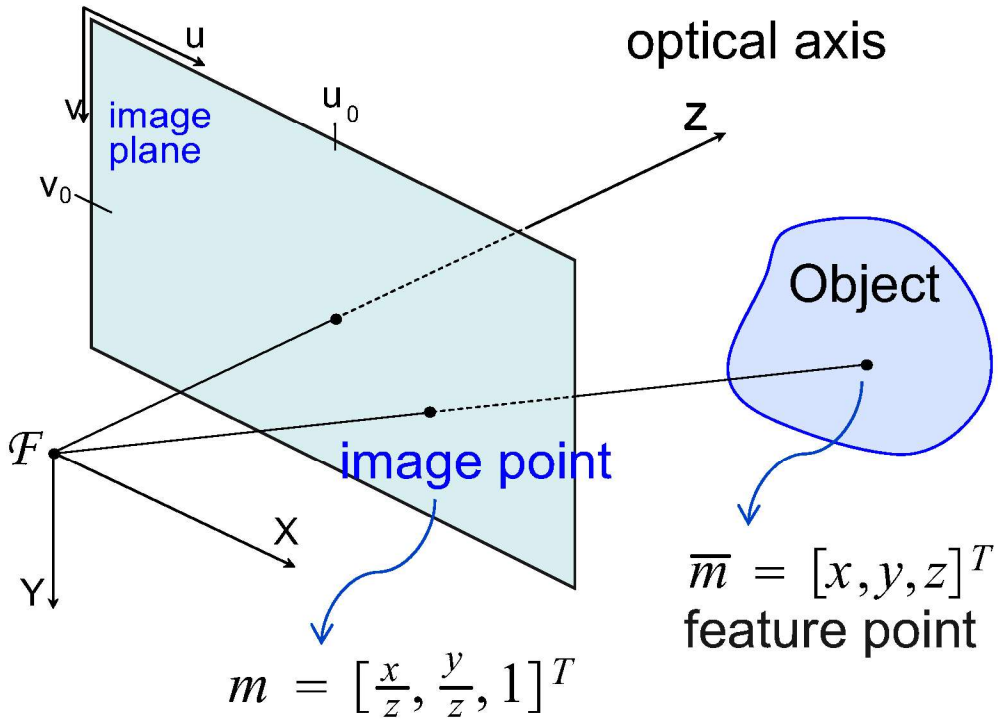
Pinhole Camera Model

- 3D feature point with Euclidean coordinates in camera frame $\mathcal{F}$

$$\bar{m} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

- Projected to image plane with normalized coordinates in $\mathcal{F}$

$$m = \begin{bmatrix} m_x \\ m_y \\ 1 \end{bmatrix} = \frac{\bar{m}}{z} = \begin{bmatrix} x/z \\ y/z \\ 1 \end{bmatrix}$$



- Mapped to pixel coordinates by Calibration Matrix

$$p = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = Am = \begin{bmatrix} f\sigma_x & -f\sigma_x \tan\alpha & u_0 \\ 0 & f\sigma_y \sec\alpha & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} m_x \\ m_y \\ 1 \end{bmatrix}$$

A: camera calibration matrix

Pinhole Camera Model (Remarks)

- Many estimation and control schemes require normalized coordinates for points
- Given pixel coordinates p of a point in the image and knowledge of calibration matrix A, recover normalized coordinates

$$m = A^{-1} p$$

- There is no way to recover the Euclidean coordinates without additional information. Depth ambiguity is a consequence of imaging.

Camera Calibration

- Consider an inertial, world reference frame $\mathcal{F}_w$
- The camera frame $\mathcal{F}_c$ has translation T and rotation R w.r.t. $\mathcal{F}_w$.
- A point has coordinates $m_w=[x_w,y_w,z_w]^T$ in the world frame.
- Pixel coordinates of the point in the image are given by

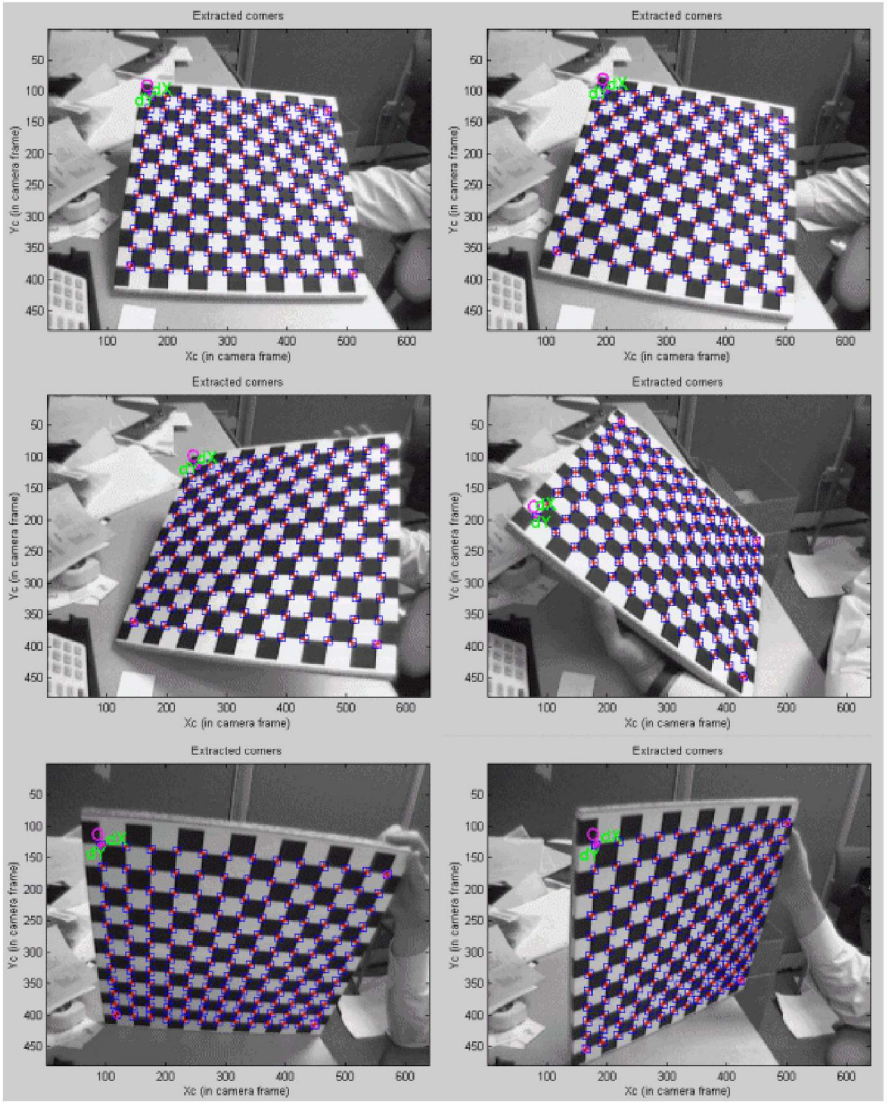
$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f\sigma_x & -f\sigma_x \tan\alpha & u_0 \\ 0 & f\sigma_y \sec\alpha & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1/z & 0 & 0 & 0 \\ 0 & 1/z & 0 & 0 \\ 0 & 0 & 1/z & 0 \end{bmatrix} \begin{bmatrix} R & T \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

Intrinsic calibration matrix — Projection matrix — Extrinsic calibration matrix

- Calibration is the processing of recovering these parameters

Camera Calibration

- Most calibration methods require some sort of known target such as a calibration board
- Numerous images of target are taken
- Features, such as corners, are extracted
- Comparison of known target and image points allows solving of intrinsic and extrinsic camera parameters
- Intrinsic calibration parameters are usually solved simply as the elements of the calibration matrix A



Lecture 3: Structure and Pose Estimation

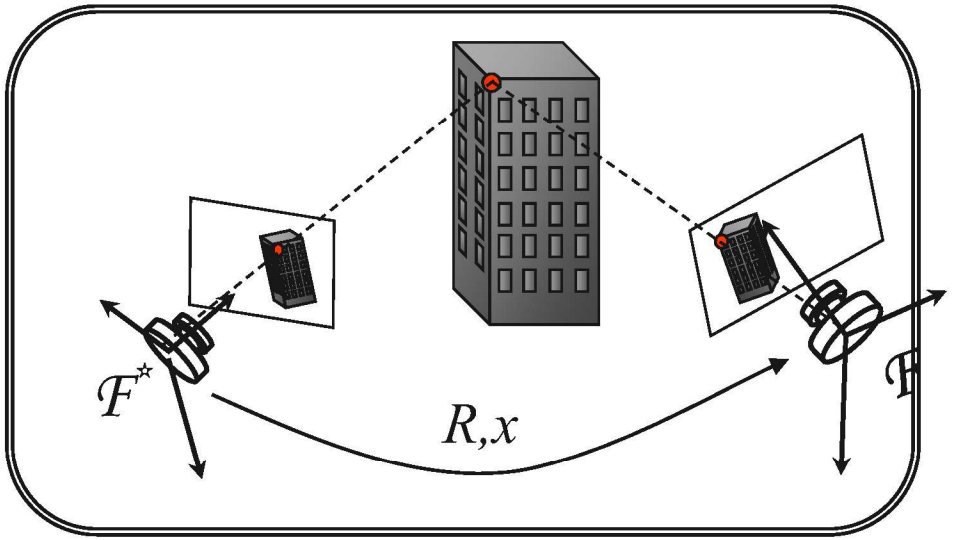
- Structure from Motion
- Pose reconstruction
- Essential matrix and 8-point algorithm
- Homography matrix and 4-point algorithm

Structure From Motion - Introduction

- Given two images of matched points, it is possible to determine:
  - The rotation and translation that separates these two camera views
  - The relative structure of the feature points being viewed
- We assume feature points have already been identified and matched between two views through feature point detection and tracking methods
- Commonly referred to as “Structure From Motion (SFM)”
- Different situations/scenes will require different methods of reconstruction
- The methods presented here are linear, in that they only require methods of linear algebra

Pose Reconstruction

- A rotation $R \in \mathrm{SO}(3)$ and translation $x \in \mathbb{R}^3$ separate the camera poses $\mathcal{F}$ and $\mathcal{F}^*$
- N feature points are extracted and matched in both images



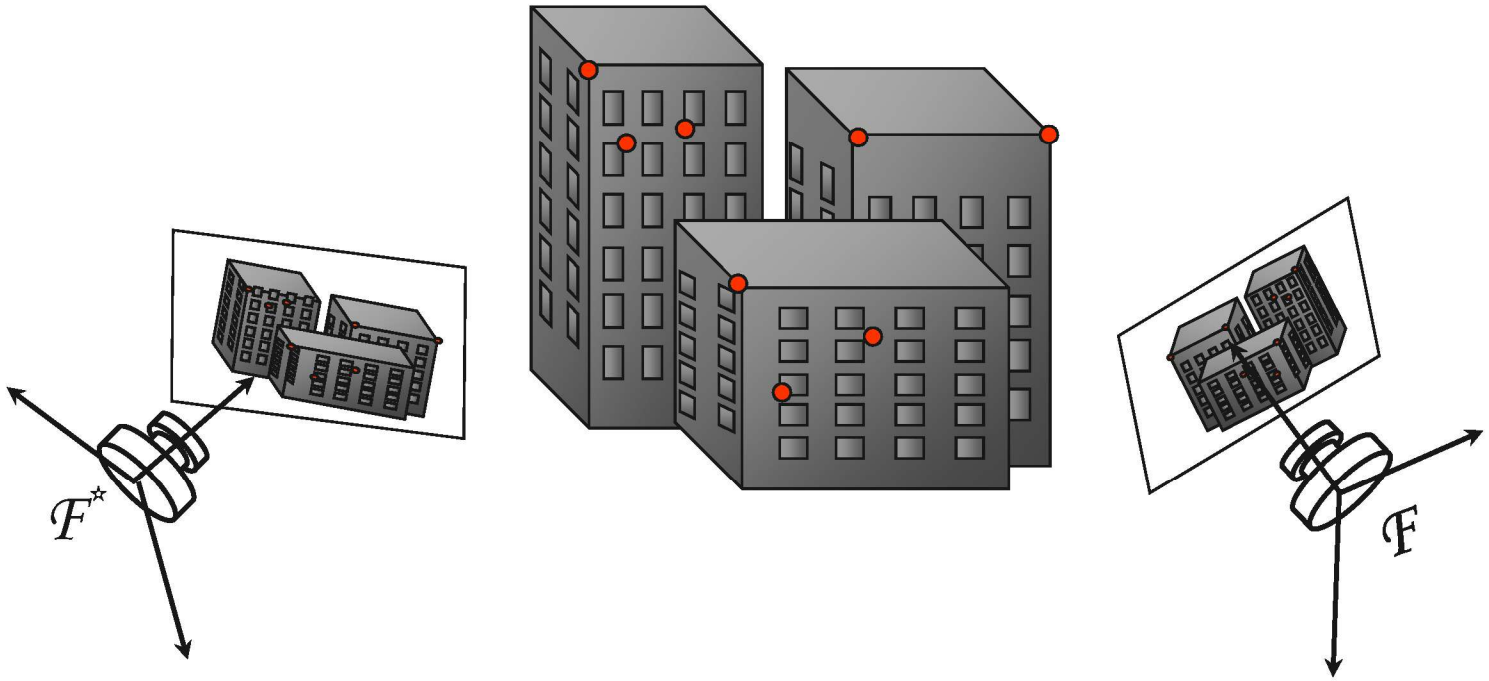
- Feature points have 3D coordinates in the two frames given by

$$\bar{m}_j^* = \begin{bmatrix} x_j^* & y_j^* & z_j^* \end{bmatrix}^T, \quad \bar{m}_j = \begin{bmatrix} x_j & y_j & z_j \end{bmatrix}^T, \quad \forall j \in \{1 \dots N\}$$

- The coordinates of each point in the camera frames are related by

$$\bar{m}_j = R\bar{m}_j^* + x, \ \forall j \in \{1 \dots N\}$$

Essential Matrix & 8 Point Algorithm



- We begin with the Essential Matrix and 8 Point Algorithm.
- Assume at least 8 points are matched in two images, no four 3D points are coplanar.

  - H. Longuet-Higgins, “A computer algorithm for reconstructing a scene from two projections,” *Nature*, pp. 133 – 135, Sept. 1981.
  - T. Huang and O. Faugeras, “Some properties of the E matrix in two-view motion estimation,” *IEEE Trans. Pattern Anal. Machine Intell.*, vol. 11, no. 12, pp. 1310–1312, 1989.

Essential Matrix & 8 Point Algorithm

- Image feature points have 2D coordinates given by

$$m_j^* = \begin{bmatrix} \frac{x_j^*}{z_j^*} & \frac{y_j^*}{z_j^*} & 1 \end{bmatrix}^T, \quad m_j(t) = \begin{bmatrix} \frac{x_j(t)}{z_j(t)} & \frac{y_j(t)}{z_j(t)} & 1 \end{bmatrix}^T, \quad \forall j \in \{1 \ldots N\}$$

- The coordinates of each point in the images are related by

$$z_j m_j = z_j^* R m_j^* + x, \ \forall j \in \{1 \ldots N\}$$

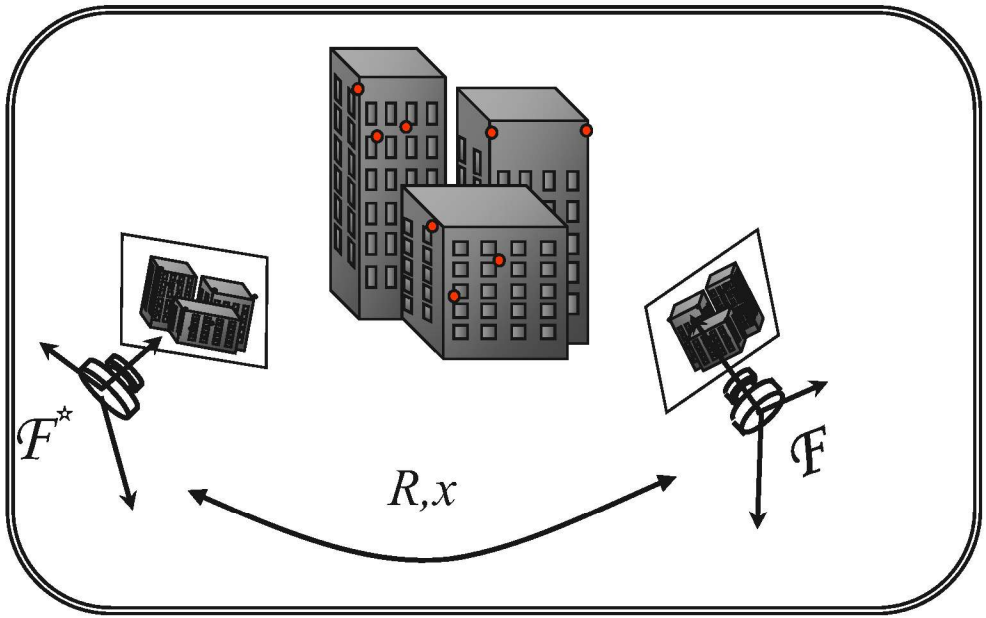
- Define $[x]_x$ as a skew-symmetric matrix such that $[x]_x m = x \times m$

$$z_j [x]_\times m_j = z_j^* [x]_\times R m_j^*$$

- Note that

$$z_j m_j^T [x]_\times m_j = 0$$

$$z_j m_j \cdot (x \times m_j) = 0$$



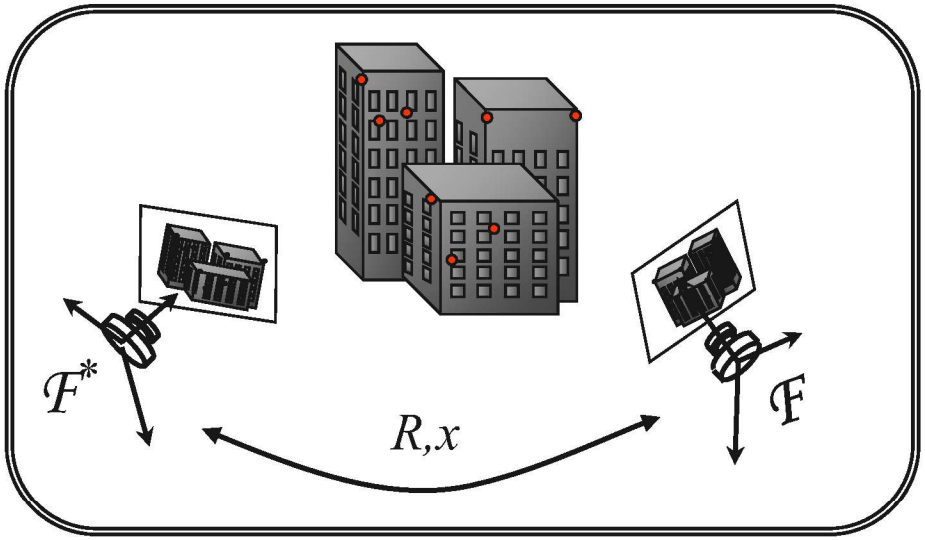
Essential Matrix & 8 Point Algorithm

- The equation

$$z_j^* m_j^T [x]_\times R m_j^* = 0$$

Loss of scale information

$$m_j^T E m_j^* = 0$$



holds for all points in the images, and is known as the Essential Constraint or Epipolar Constraint.

- $E = [x]_\times R$ is the Essential Matrix and contains relative pose information

Essential Matrix & 8 Point Algorithm

- If eight or more points are available, E can be estimated using linear algebra methods. This is known as the Eight Point Algorithm.
- "Unwrap" $E \in \mathbb{R}^{3x3}$ to a vector $e \in \mathbb{R}^9$

$$E = \begin{bmatrix} e_1 & e_2 & e_3 \\ e_4 & e_5 & e_6 \\ e_7 & e_8 & e_9 \end{bmatrix}^T \Rightarrow e = \begin{bmatrix} e_1, & e_2, & e_3, & e_4, & e_5, & e_6, & e_7, & e_8, & e_9 \end{bmatrix}^T$$

- Define a row vector M_j for the jth feature point

$$M_j = [m_x^* m_x, m_x^* m_y, m_x^*, m_y^* m_x, m_y^* m_y, m_y^*, m_x, m_y, 1]$$

- The Epipolar constraint can be rewritten as

$$M_j e = 0$$

- Concatenate all N vectors M_j to form matrix $M \in \mathbb{R}^{Nx9}$

$$Me = 0$$

Essential Matrix & 8 Point Algorithm

- The null space of M will give a vector e that satisfies the equation

$$Me = 0$$

- "Wrap" e back into a 3x3 matrix to get estimate of E

  Note that E will only be estimated up to a scale factor

- Noise and calibration errors tend to make the estimate of E incorrect, and won't necessarily satisfy $E = [x]_{\times} R$
- Any matrix that satisfies the above will have two identical singular values and one zero singular value
- We project our estimate of E to the closest true Essential Matrix
- Take SVD of E such that $E=U\Sigma V^T$, with $\Sigma=diag\{\sigma_1,\sigma_2,\sigma_3\}$
- Replace Σ with $\Sigma'=diag\{1,1,0\}$

Essential Matrix & 8 Point Algorithm

- With E now estimated, how do we recover x and R?
- Define a rotation matrix

$$R_z(\pm\tfrac{\pi}{2}) = \begin{bmatrix} 0 & \mp 1 & 0 \\ \pm 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Having taken the SVD of E such that $E=U\Sigma V^T$, with $\Sigma=diag\{1,1,0\}$, it can be shown that

$$[x']_\times = UR_z(\pm\tfrac{\pi}{2})\Sigma U^T \qquad R = UR_z(\pm\tfrac{\pi}{2})V^T$$

Two Solutions!!

where $[x']_x = \lambda\,[x]_x$ for some unknown scale factor λ

Essential Matrix & 8 Point Algorithm

- It gets worse, note that the essential constraint $m^T E m^* = 0$ is valid for $\pm E$
- We need to repeat the previous analysis with $\Sigma = diag\{-1,-1,0\}$
- This gives four possible solutions for the pose

- Luckily, three solutions will be physically impossible as they would require at least some feature points to be behind the camera
- For all points, test the equation $\lambda m_j = \lambda^* R m_j^* + x'$

it should be that for 3 solutions, $\lambda<0$, $\lambda^*<0$, or both to be satisfied

We have thus recovered rotation R and scaled translation $x' = \lambda' x$

Essential Matrix & 8 Point Algorithm

- With R and x' known, we can recover the relative depths of the feature points up to a scale factor λ'
- Manipulate the rigid body equation

$$\lambda m_j = \lambda^* R m_j^* + x'$$

$$\lambda^* [m_j]_\times R m_j^* + [m_j]_\times x' = 0$$

$$\begin{bmatrix} [m_j]_\times R m_j^* & [m_j]_\times x' \end{bmatrix} \begin{bmatrix} \lambda_j^* \\ 1 \end{bmatrix} = 0$$

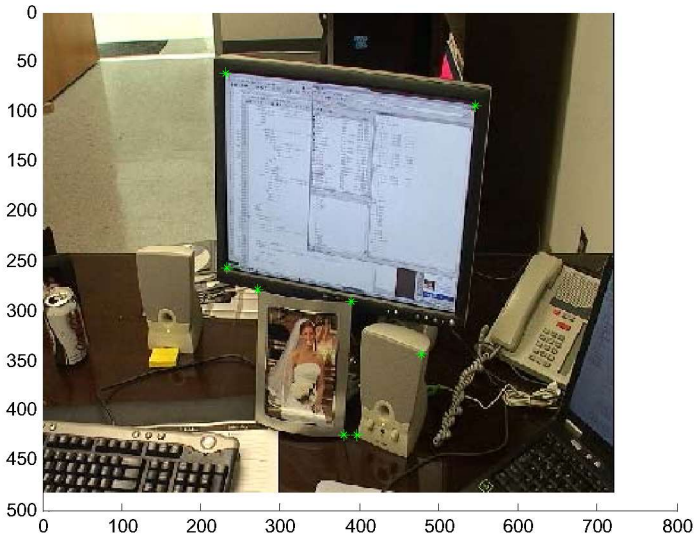
- Combined Matrix for all feature points

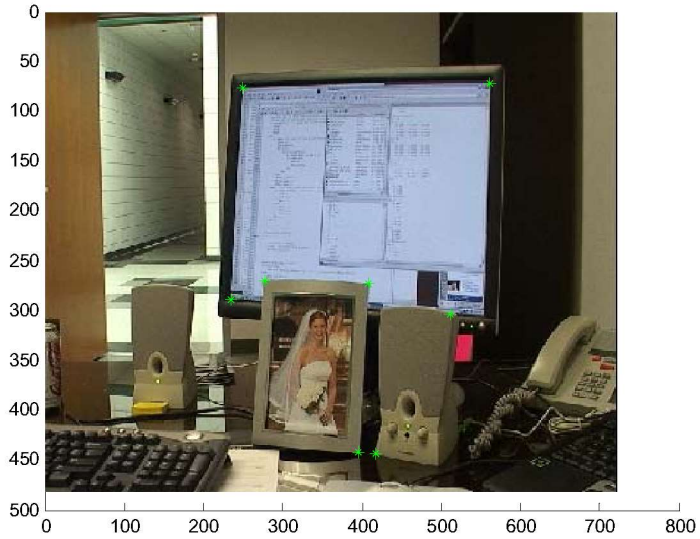
$$\begin{bmatrix} [m_1]_\times R m_1^* & 0 & \cdots & 0 & [m_1]_\times x' \\ 0 & [m_2]_\times R m_2^* & \cdots & 0 & [m_2]_\times x' \\ \vdots & \vdots & \ddots & 0 & [m_{N-1}]_\times x' \\ 0 & 0 & 0 & [m_N]_\times R m_N^* & [m_N]_\times x' \end{bmatrix} \begin{bmatrix} \lambda^* \\ 1 \end{bmatrix} = 0$$

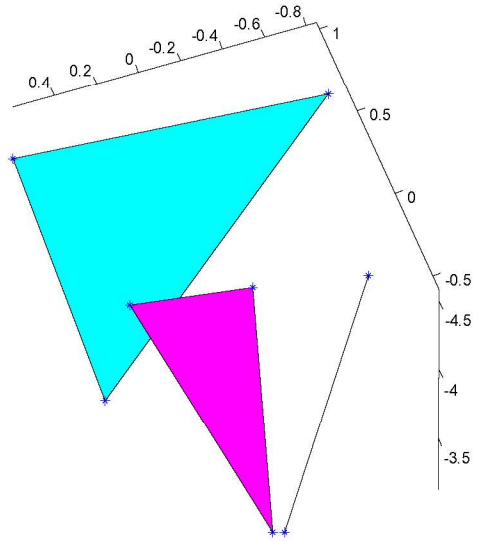
$3N \times N+1$ $\qquad$ $N+1$

Null Space gives all depths up to scale factor

Example of 8 Point Algorithm







$$x4 = \begin{bmatrix} -0.1165 \\ 0.9144 \\ 0.3877 \end{bmatrix} \qquad R4 = \begin{bmatrix} 0.9928 & -0.0948 & 0.0727 \\ 0.1090 & 0.9683 & -0.2248 \\ -0.0491 & 0.2311 & 0.9717 \end{bmatrix}$$

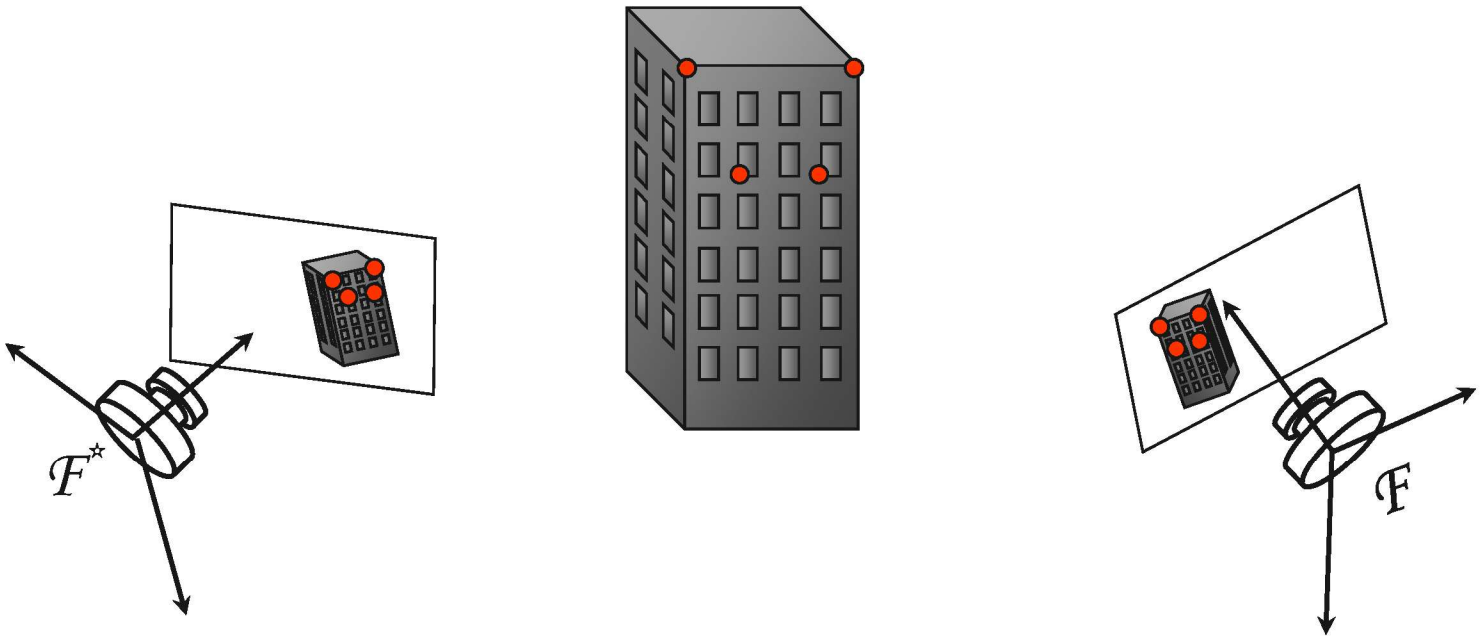
Translation and rotation seem like good match

Reconstructed points are reasonable

Homography Matrix

- **Homography**: The geometric concept of homography is a one-to-one and on-to transformation or mapping between two sets of points. In computer vision, homography refers to the mapping between points in two Euclidean-planes (Euclidean homography), or to the mapping between points in two images (projective homography).

Multi-View Geometry



- We now address the issue of planar scenes using the Euclidean Homography Matrix and the 4 point algorithm
- Assume at least 4 coplanar, noncollinear points are matched in two images

O. Faugeras and F. Lustman, "Motion and structure from motion in a piecewise planar environment," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 2, no. 3, pp. 485–508, 1988.

Euclidean Geometry

- Image features points have 2D coordinates given by

$$m_j^* = \begin{bmatrix} \frac{x_j^*}{z_j^*} & \frac{y_j^*}{z_j^*} & 1 \end{bmatrix}^T, \quad m_j(t) = \begin{bmatrix} \frac{x_j(t)}{z_j(t)} & \frac{y_j(t)}{z_j(t)} & 1 \end{bmatrix}^T, \quad \forall j \in \{1 \ldots N\}$$

- The coordinates of each point in the images are related by

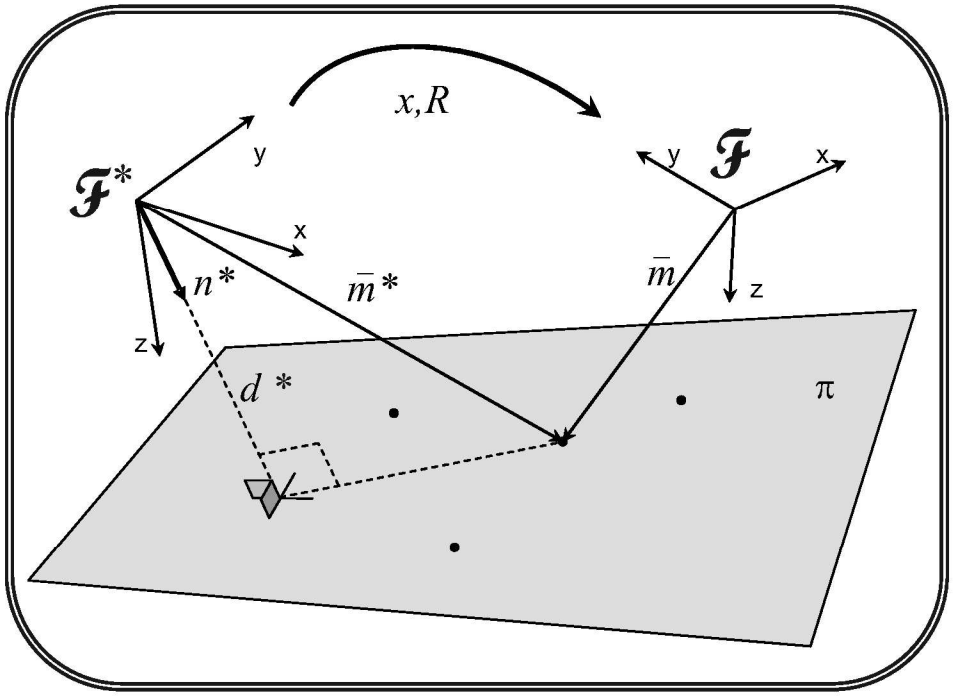
$$\overline{m}_j(t) = R(t)\overline{m}_j^* + x(t)$$

$$z_j m_j = z_j^* R m_j^* + x, \ \forall j \in \{1 \ldots N\}$$

- The points all lie in a plane π, which has normal vector n^* in $\mathcal{F}^*$, and lies a distance d^* from the origin

- The following relationship is true for all $\overline{m}_j^*$

$$d^* = n^{*T}\overline{m}_j^*$$



Euclidean Geometry & Homography Matrix

- Combine equations

$$\overline{m}_j(t) = R(t)\overline{m}_j^* + x(t)$$

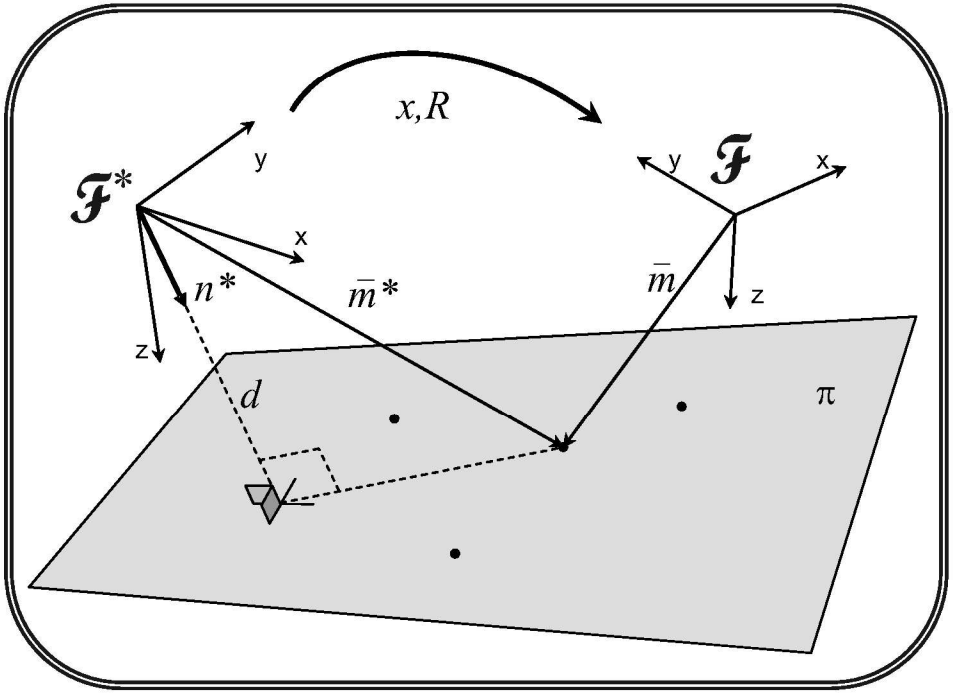
$$d^* = n^{*T}\overline{m}_j^*$$

to get

Loss of scale information

$$m_j = \frac{z_j^*}{z_j}\left(R + \frac{x}{d^*}n^{*T}\right)m_j^*$$

$$= \alpha_j H m_j^*$$



- $H = R + \frac{x}{d^*}n^{*T}$ is the **Euclidean Homography Matrix** that contains relative pose information and some structure information

- $\alpha_j = \frac{z_j^*}{z_j}$ is a ratio of depths (called depth ratio)

Homography Decomposition

- If four or more points are available, H can be estimated up to a scale factor using linear algebra methods (we recover scale later).
- Scale H by its 3,3 element to get H_n
- "Unwrap" $H_n \in \mathbb{R}^{3x3}$ to a vector $h \in \mathbb{R}^8$

$$H_n = \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & 1 \end{bmatrix} \Longrightarrow h = [h_1, h_2, h_3, h_4, h_5, h_6, h_7, h_8]^T$$

- Define a $2x8$ matrix M_j for the j^{th} feature point

$$M_j = \begin{bmatrix} m_x^*, m_y^*, 1, 0, 0, 0, -m_x^* m_x, -m_y^* m_x \\ 0, 0, 0, m_x^*, m_y^*, 1, -m_x^* m_y, -m_y^* m_y \end{bmatrix} \text{ such that } M_j h = m_j$$

- Concatenate all N vectors m_j and matrices M_j to form matrix $M \in \mathbb{R}^{2N x 8}$

$$Mh = m$$

Homography Decomposition (Cont'd)

- Use standard linear algebra to find vector h that satisfies

$$Mh = m$$

- Wrap h back into a 3x3 matrix to get estimate of H_n
- Take the SVD of H_n such that $H_n = U\Sigma V^T$, this maps to an orthogonal space such that

$$\Sigma = R' + x' n'^T$$

- If we find R', x', n', we can recover R, x/d^*, n^* as

$$R = UR'V^T \qquad \frac{x}{d^*} = Ux' \qquad n^* = Vn'$$

- Recovery of these terms gets pretty complicated

Homography Decomposition (Cont'd)

- First recover the lost scale of H, i.e. find h_{33}
- $\Sigma = diag\{\sigma_1, \sigma_2, \sigma_3\}$,
- $h_{33} = \pm\sigma_2$ such that $det(U)det(V)\sigma_2 > 0$
- There are different solutions depending on number of unique singular values, and the sign of $det(U)det(V)$

- If all singular values are the same, there was no translation, so

$$R = H,\ x = 0,\ n^* \text{ is unknown}$$

$$H = R + \frac{x}{d^*} n^{*T}$$

Homography Decomposition (Cont'd)

- Recover $H = H_n h_{33}$
- Recover $\alpha_j = 1/(H_3^T m_j)$ where H_3^T is the third row of H
- Test each solution for positive depth constraint

$$n^{*T}\overline{m}_j^* = d^* > 0 \quad \Longrightarrow \quad n^{*T} m_j > 0 \;\; \forall j$$

- In the general case, two solutions will pass the depth constraint, how to choose the correct one?
  - A priori knowledge of the scene can be used to test n^* vs expected normal
  - Coherence of multiple views can be used. Solve for solutions to a third view with respect to $\mathcal{F}^*$, pick the solutions that have same (or closest) n^*

Homography Decomposition (Cont'd)

- We have thus recovered rotation R and scaled translation $x' = x/d^*$
- Can recover relative feature point depths as of the feature points up to a scale factor by noting

$$\bar{m}_j^T n^* = d^*$$

$$z_j m_j^T n^* = d^*$$

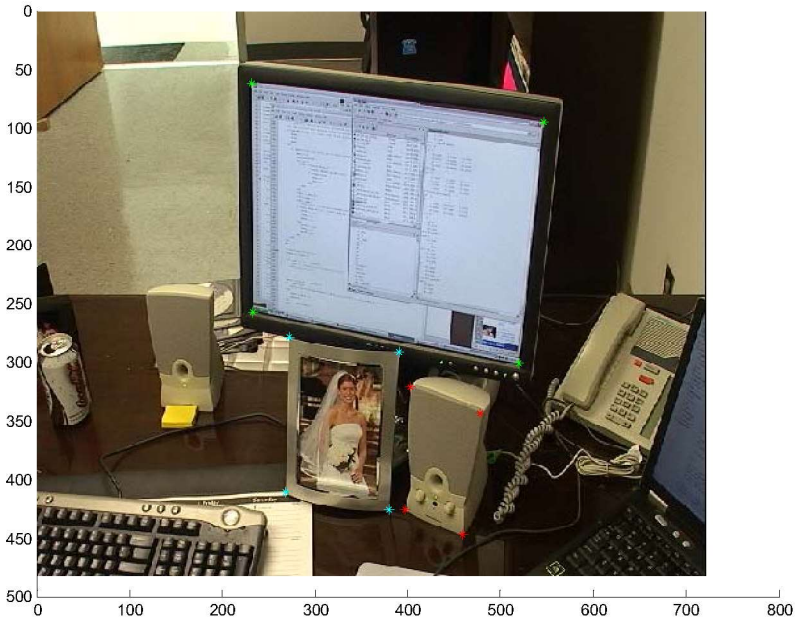
$$\lambda_j m_j^T n^* = 1$$

$$\lambda_j = \frac{1}{m_j^T n^*}$$

- If d^* is known or can be measured or estimated, we can recover translation and Euclidean coordinates of all points

Example of 4 Point Algorithm

Example using the computer vision toolbox



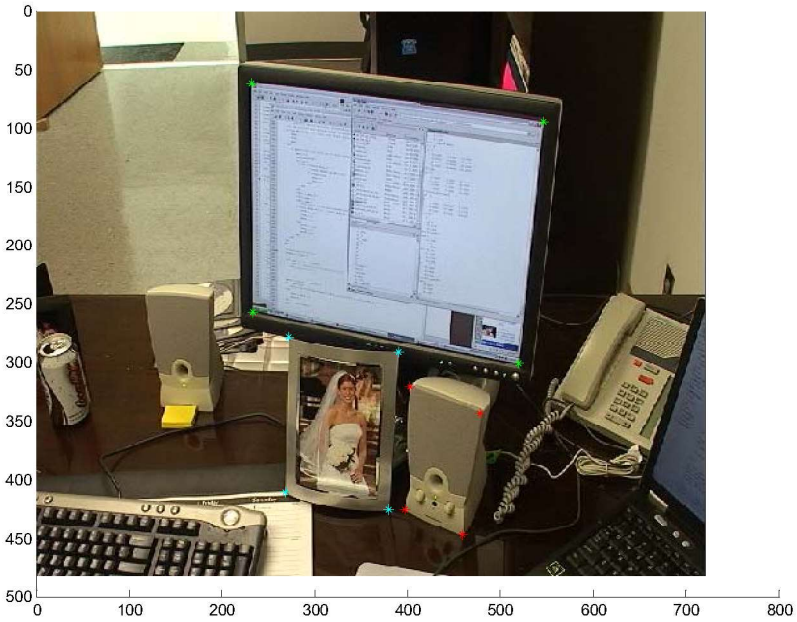


Image at $\mathcal{F}^*$

Image at $\mathcal{F}$

A third image was taken for coherence

Lec 4: Kalman Filter and Sensor Integration

- Expectation, variance, co-variance
- Kalman Filter
- Sensor integration

Kalman Estimation

KALMAN Estimation: Given a system represented as a linear discrete Markov process, the state-space model

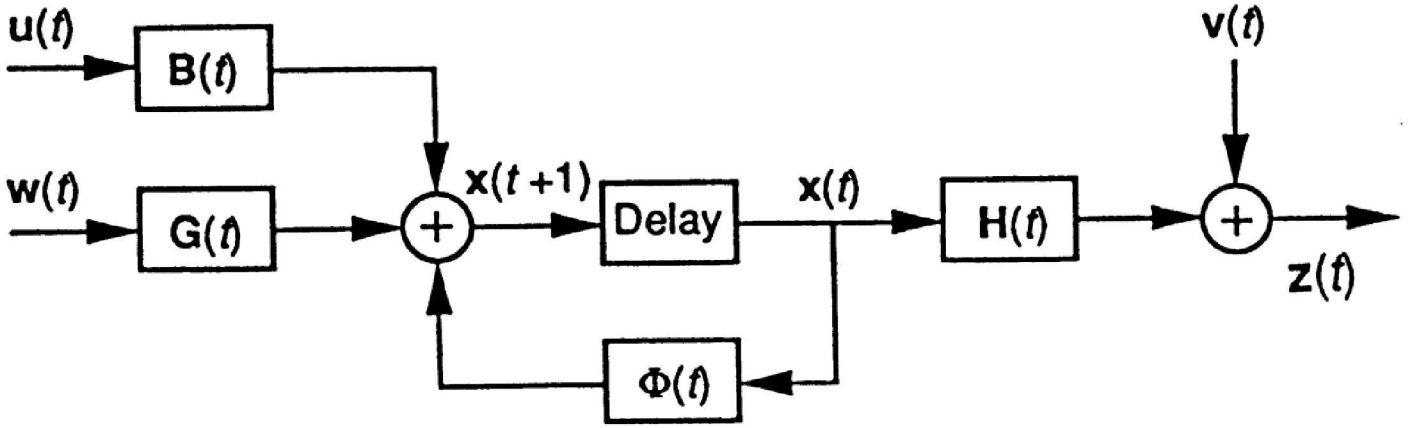
$$x(t+1) = \Phi(t)x(t) + B(t)u(t) + G(t)w(t) \qquad (2)$$

Zero-mean Gaussian noise

and the measurement model

$$z(t) = H(t)x(t) + v(t) \qquad (3)$$

can be used to describe the system below, where



x : m state vector

Φ : m × m state transition matrix

B : m × p input transition matrix

u : p input vector (e.g., position of sensor platform)

G : m × q process noise transmission matrix

w : q process noise vector

z : n measurement vector

H : n × m measurement matrix

v : n measurement noise vector

The *w* and *v* are assumed uncorrelated discrete-time ***zero-mean*** white Gaussian noise sequences with covariance kernels:

$$E\{w(t_i)w^T(t_j)\} = Q(t_i)\delta_{ij} \quad (4)$$

$$E\{v(t_i)v^T(t_j)) = R(t_i)\delta ij \quad (5)$$

where E{· · ·} denotes the expectation operator and the Kronecker delta function.

Kalman Filtering Equations

When all of the parameters (the matrices Φ, B, G, H, Q and R) of the models are known, the optimal *Kalman filtering* equations are:

$$\hat{X}(t|t) = \hat{x}(t|t-1) + K(t)[z(t) - H(t)\hat{x}(t|t-1)] \quad (6)$$

$$\hat{x}(t+1|t) = \Phi(t)\hat{x}(t|t) + B(t)u(t) \quad (7)$$

where $\hat{x}(t\,|\,t)$ is the estimate of $x(t)$ based on the measurements $\{z(0), \cdots, z(t)\}$,

and $\hat{x}(t+1\,|\,t)$ is the prediction of $x(t+1)$ based on the measurements $\{z(0), \cdots, z(t)\}$.

The m× n matrix *K* is the "Kalman filter gain" and is computed by

$$K(t) = P(t|t-1)H^T(t)[H(t)P(t|t-l)H^T(t) + R(t)]^{-1} \quad (8)$$

where $P(t|t-1) = E\{(x(t) - \hat{x}(t|t-1))(x(t) - \hat{x}(t|t-1))^T\}$

is the m×n conditional covariance matrix of the error in predicting *x*(*t*) and is determined using

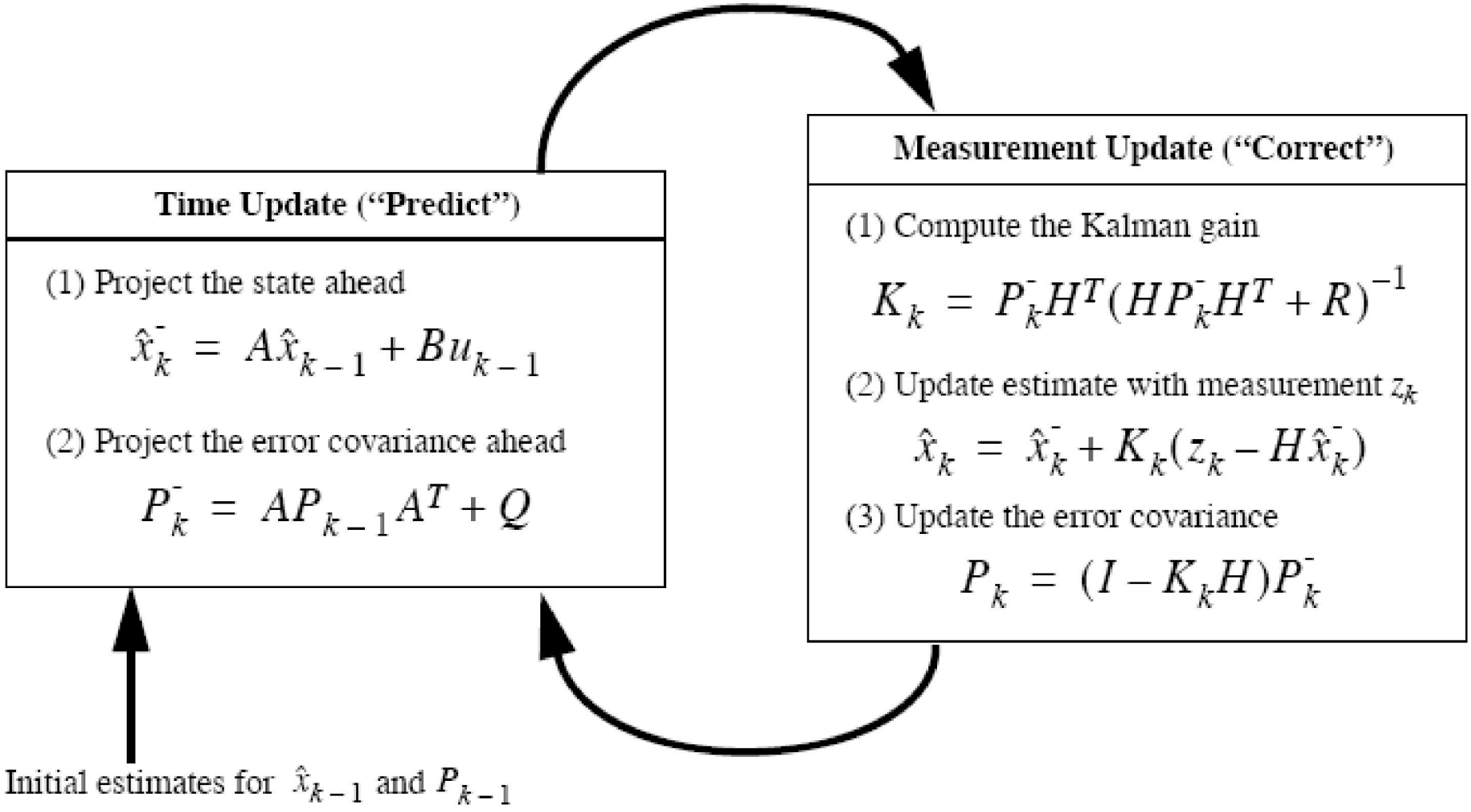
$$P(t+1|t) = \Phi(t)P(t|t)\Phi^T(t) + G(t)Q(t)G^T(t) \quad (9)$$

where $P(t|t) = P(t|t-1) - K(t)H(t)P(t|t-1)$

The initial conditions for the recursion are given by $\hat{x}(0|0) = \hat{x_0}$ and $P(0|0) = P_0$.

Sequential State Estimation

Sequential State Estimation by Kalman Filtering



Derivation of the Kalman Gain

Derivation of the Kalman Gain for the scalar case

Given : $x_{k+1} = Ax_k + Bu_k + w_k$ --- (a)

and the measurement model

$$z_k = H x_k + v_k \quad \text{--- (b)}$$

where w_k and v_k are gaussian white noise – processes with zero mean and variances Q and R respectively. Assume also that the initial state $x(0)$ is Gaussian and $E\{x(0)\} = m_0$ and $\text{cov}\{x(0)\} = R_0$.

Q, R and R_0 are non-negative scalars.

The Kalman estimate of x is of the form :

$$\hat{x}_{k+1} = A\,\hat{x}_k + Bu_k + K_k[z_k - H\,\hat{x}_k] \qquad ---(c)$$

The Kalman filter design principle is to ***minimize the variance*** of the new state estimate.

$$\tilde{x}_{k+1} = x_{k+1} - \hat{x}_{k+1} \qquad ---(d)$$

$$\tilde{x}_{k+1} = \{A\,x_k + Bu_k + w_k\} - \{A\,\hat{x}_k + Bu_k + K_k[z_k - H\,\hat{x}_k]\}$$

$$= A\,x_k + Bu_k + w_k - \{A\,\hat{x}_k + Bu_k + K_k[H\,x_k + v_k - H\,\hat{x}_k]\}$$

$$= A\,(x_k - \hat{x}_k) + w_k - K_k[H\,(x_k - \hat{x}_k) + v_k]$$

$$= A\,(\tilde{x}_k) + w_k - K_k[H\,\tilde{x}_k + v_k] \qquad ---(e)$$

$$P_{k+1} = V(\tilde{x}_{k+1}) = E(\tilde{x}_{k+1}^{\,2}) - E(\tilde{x}_{k+1})^2$$
$$= E(\tilde{x}_{k+1}^{\,2}) \qquad since \quad E(\tilde{x}_{k+1}) = 0 \quad is\ assumed.$$

$$P_{k+1} = E(\tilde{x}_{k+1}^2) = E\{(A\,\tilde{x}_k + w_k - K_k H\,\tilde{x}_k - K_k v_k\,])^2\}$$
$$= E\{([A - K_k H]\,\tilde{x}_k + w_k - K_k v_k)^2\}$$
$$= E\{[A - K_k H]^2\,\tilde{x}_k^2\} + E\{w_k^2\} + E\{K_k^2 v_k^2\}$$

In the above expansion, note that the expectation of terms containing ($\tilde{x}$, w), ($\tilde{x}$, v) and (w, v) are zero as they are assumed to be uncorrelated.

$$P_{k+1} = [A - K_k H]^2 \, E(\tilde{x}_k^2) + E(w_k^2) + K_k^2 \, E(v_k^2)$$

$$= [A - K_k H]^2 \, P_k + Q + K_k^2 \, R \qquad ---(f)$$

Kalman minimizes P_{k+1} with respect to K_k, the Kalman gain.

$$\frac{\partial P_{k+1}}{\partial K_k} = \frac{\partial}{\partial K_k}\left([A - K_k H]^2 P_k + Q + K_k^2 R \right)$$

$$= -2H[A - K_k H]P_k + 2K_k R = 0$$

$$HAP_k - K_k H^2 P_k = K_k R$$

$$[H^2 P_k + R] K_k = HAP_k$$

And finally, $$K_k = \frac{HAP_k}{H^2 P_k + R}$$

We could also find K in another way. From Equation (f) :

$$P_{k+1} = [A - K_k H]^2 P_k + Q + K_k^2 R$$

$$= [A - K_k H]^2 P_k + Q + K_k^2 R$$

In Equation g, P_{k+1} is minimized w.r.t. K_k by setting K_k as

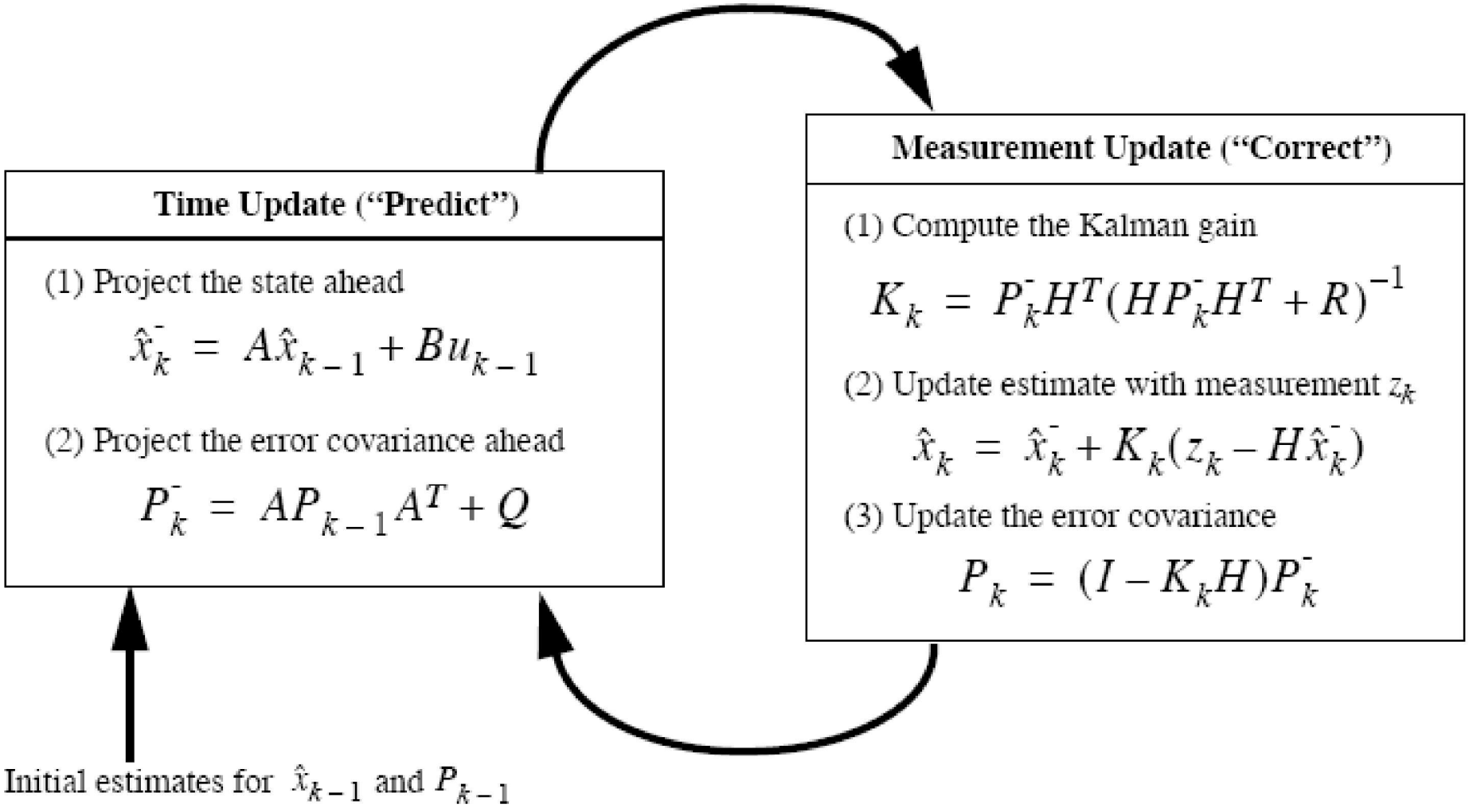
$$K_k = \frac{AHP_k}{H^2 P_k + R}$$

So, we obtain the same result as by the differentiation approach.

Now. the variance P_{k+1} becomes :

$$P_{k+1} = -\frac{(AHP_k)^2}{H^2 P_k + R} + Q + A^2 P_k$$

Sequential state estimation by Kalman Filtering



Kalman Filter for Sensor Fusion

Sensor information from different sensors can be fused by Kalman filter.

Example : The application of Kalman filtering for multi-sensor fusion can be illustrated using the object recognition of a landmark. Sensors 1 and 2, respectively S1 and S2, provide redundant information relative to each other concerning the shape of the objects to be recognized. The state to be estimated is the shape x of an object and can be assumed to ***remain constant*** over time, i.e., $x(t) = x$ for all t.

$$x(t+1) = x(t) \qquad \text{i.e. } \Phi \text{ (or } A) = 1$$

x(t+1) = x(t) shape (state) of object don't change

The shape measurements z_1 and z_2 from S1 and S2, respectively, can be modeled as

$$\begin{aligned} z_1 &= x + v_1 \\ z_2 &= x + v_2, \end{aligned} \qquad (10)$$

where v_1 and v_2 are independent zero-mean Gaussian random variables with variances σ_1^2 and σ_2^2, respectively.

We can consider ***two possible cases***. The two sensor information can be processed 1) simultaneously (batch) or 2) sequentially.

1) Batch Processing

If the measurements from S1 and S2 are ***available simultaneously***, batch processing can be used for fusion, where

$$z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} x + \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = Hx + v$$

Thus we have

$$x(t+1) = x(t)$$

$$z_1 = x + v_1$$

$$z_2 = x + v_2,$$

Shape does not change, so state x is constant.

where v_1 and v_2 are independent zero-mean Gaussian random variables with variances σ_1^2 and σ_2^2, respectively.

The Kalman estimate of x is of the form :

$$\hat{x}_{k+1} = \hat{x}_k + K_k[z_k - H\hat{x}_k]$$
$$= \hat{x}_k + K_1(z_1 - \hat{x}_k) + K_2(z_2 - \hat{x}_k)$$

The Kalman filter design principle is to ***minimize the variance*** of the new state estimate error. The state estimate error is given by

$$\tilde{x}_{k+1} = x_{k+1} - \hat{x}_{k+1}$$
$$error \quad actual \quad extimated$$

Now,

$$\tilde{x}_{k+1} = x_k - [\hat{x}_k + K_1(z_1 - \hat{x}_k) + K_2(z_2 - \hat{x}_k)]$$
$$= x_k - \hat{x}_k - [K_1(z_1 - \hat{x}_k) + K_2(z_2 - \hat{x}_k)]$$

Q : We could also try for a slightly less general approach with only one Gain for both residuals : $\hat{x}_{k+1} = \hat{x}_k + K[(z_1 - \hat{x}_k) + (z_2 - \hat{x}_k)]$

$$\tilde{x}_{k+1} = (x_k - \hat{x}_k) - [K_1(x_k + v_1 - \hat{x}_k) + K_2(x_k + v_2 - \hat{x}_k)]$$

$$= \tilde{x}_k - [K_1(\tilde{x}_k + v_1) + K_2(\tilde{x}_k + v_2)]$$

$$= [1 - K_1 - K_2]\tilde{x}_k - K_1 v_1 - K_2 v_2$$

Now the variance of $\tilde{x}_{k+1}$ is

$$P_{k+1} = V(\tilde{x}_{k+1}) = E(\tilde{x}_{k+1}^2) - E(\tilde{x}_{k+1})^2$$

$$= E(\tilde{x}_{k+1}^2) \qquad \textit{since } E(\tilde{x}_{k+1}) = 0 \textit{ is assumed.}$$

$$P_{k+1} = E(\tilde{x}_{k+1}^{\,2})$$

$$= E\{([\,1 - K_1 - K_2\,]\,\tilde{x}_k - K_1 v_1 - K_2 v_2)^2\}$$

$$= [\,1 - K_1 - K_2\,]^2 E(\tilde{x}_k^{\,2}) + K_1^{\,2} E(v_1^{\,2}) + K_2^{\,2} E(v_2^{\,2})$$

In the above, we have assumed that the state estimate error and the noise terms v_1 and v_2 are all uncorrelated.

Therefore all the cross-correlation terms are zero.

$$P_{k+1} = [\,1 - K_1 - K_2\,]^2 P_k + K_1^{\,2}\sigma_1^{\,2} + K_2^{\,2}\sigma_2^{\,2}$$

Thus the variance of the state estimation P_{k+1} error is updated from current P_k, K_1, K_2, σ_1 and σ_2.

Next step in Kalman prediction is to minimize P_{k+1} wrt K_1 and K_2.

$$\frac{\partial P_{k+1}}{\partial K_1} = \frac{\partial}{\partial K_1}\{[\,1-K_1-K_2\,]^2 P_k + K_1^2\sigma_1^2 + K_2^2\sigma_2^2\} = 0$$

$$-2[\,1-K_1-K_2\,]P_k + 2K_1\sigma_1^2 = 0$$

Rearranging, yields

$$-[\,1-K_1-K_2\,]P_k + K_1\sigma_1^2 = 0 \qquad ---(1)$$

Similarly,

$$\frac{\partial P_{k+1}}{\partial K_2} = \frac{\partial}{\partial K_2}\{[\,1-K_1-K_2\,]^2 P_k + K_1^2\sigma_1^2 + K_2^2\sigma_2^2\} = 0$$

$$-[\,1 - K_1 - K_2\,]\,P_k + K_2\sigma_2^{\,2} = 0 \qquad ---(2)$$

Equation 1 – equation 2 yields

$$K_1\sigma_1^{\,2} = K_2\sigma_2^{\,2} \qquad ---(3)$$

square all the sigmas

Solving,

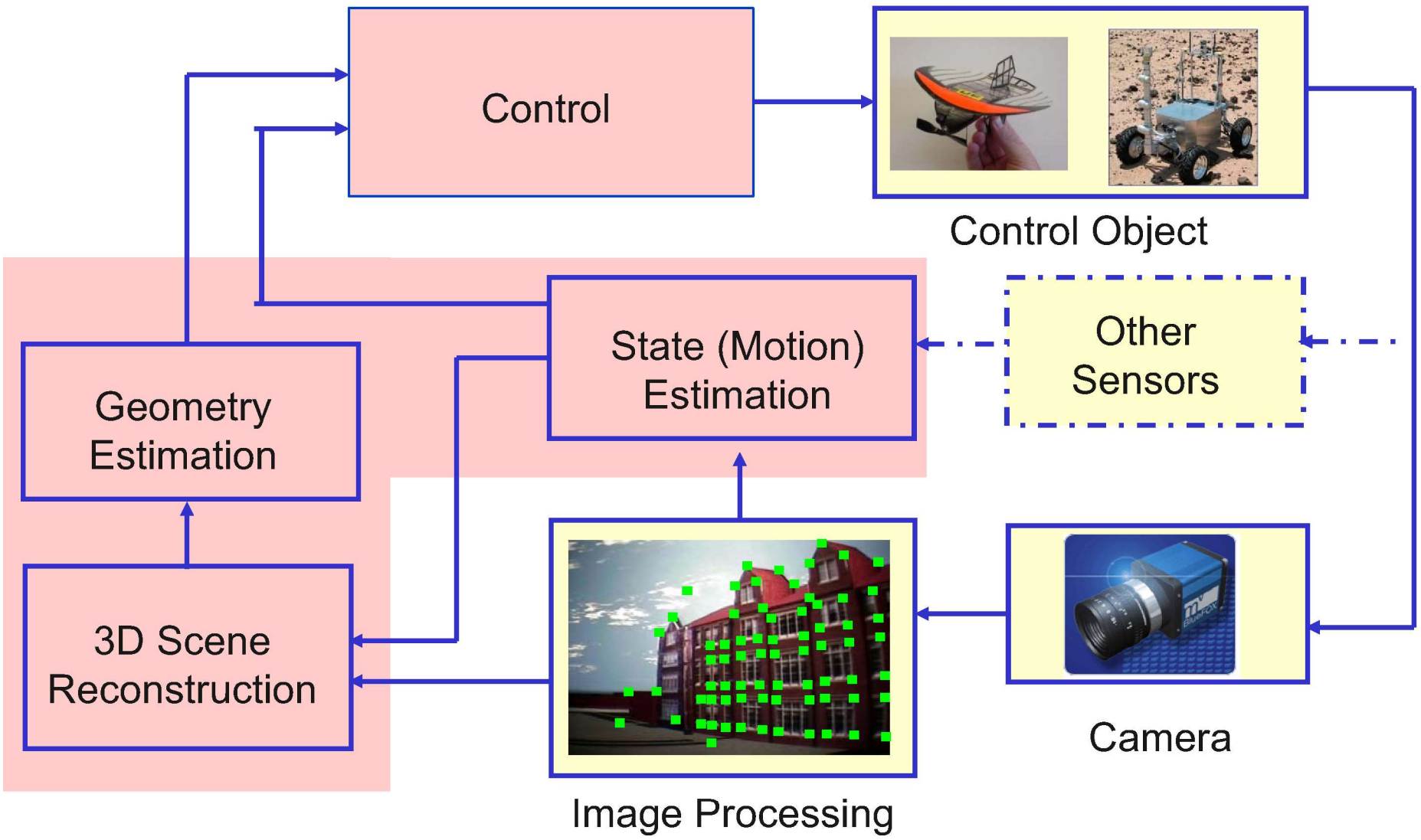
$$K_1 = \frac{\sigma_2^{\,2} P_k}{\sigma_1^{\,2} P_k + \sigma_2^{\,2} P_k + \sigma_1^{\,2}\sigma_2^{\,2}}$$

$$K_2 = \frac{\sigma_1^{\,2} P_k}{\sigma_1^{\,2} P_k + \sigma_2^{\,2} P_k + \sigma_1^{\,2}\sigma_2^{\,2}}$$

Lec 5-6: Vision-Based Control & Estimation

- Visual servo control introduction
- Three visual servo control (visual servoing) methods
  - Position-based visual servo control
  - Image-based visual servo control
  - Homography-based visual servo control (also called 2.5D visual servo control)

Vision-Based Control & Estimation



Visual Servoing-Introduction

- Visual servo control (visual servoing) is the use vision/image data in the feedback control of a mechanical system
- Typically refers to control of robot manipulators or mobile robots
- There are two broad categories, Position Based Visual Servoing (PBVS) and Image Based Visual Servoing (IBVS).
  - PBVS uses pose reconstruction methods to estimate a pose error between the current robot pose and a known goal pose.
  - IBVS regulates an error between the current image and a known goal image
  - IBVS and PBVS have strengths and weaknesses
- There are many approaches that build upon classic IBVS, PBVS or both to address unique problems, but there is no “silver-bullet” visual servoing method that is best for all situations

Visual Servoing-Introduction

- We will discuss basic PBVS and IBVS and contrast their performance.
- We will present a well known hybrid PBVS/IBVS method known as 2.5D VS that was designed to get the best of both.
- The methods presented here assume a camera mounted on a fully actuated 6DOF manipulator, the “eye-in-hand” scenario.
  - A camera mounted on a mobile robot is similar, but will likely have nonholonomic motion constraints to deal with.
  - A fixed camera viewing a manipulator or mobile robot can use PBVS or IBVS with a simple transformation from camera frame to robot frame.
- We assume the goal is a constant pose, the case of a time varying pose can be handled through standard trajectory tracking.
- We assume feature points can be extracted and tracked via feature detection and tracking methods.

Imaging Background

The Euclidean coordinates of the target points can be expressed in camera goal frame $\mathcal{F}_c^*$ and current camera frame $\mathcal{F}_c$

$$\overline{m}_j(t) = [x_j(t), y_j(t), z_j(t)]^T$$

$$\overline{m}_j^* = [x_j^*, y_j^*, z_j^*]^T$$

$$m_j(t) = [\frac{x_j(t)}{z_j(t)}, \frac{y_j(t)}{z_j(t)}, 1]^T$$

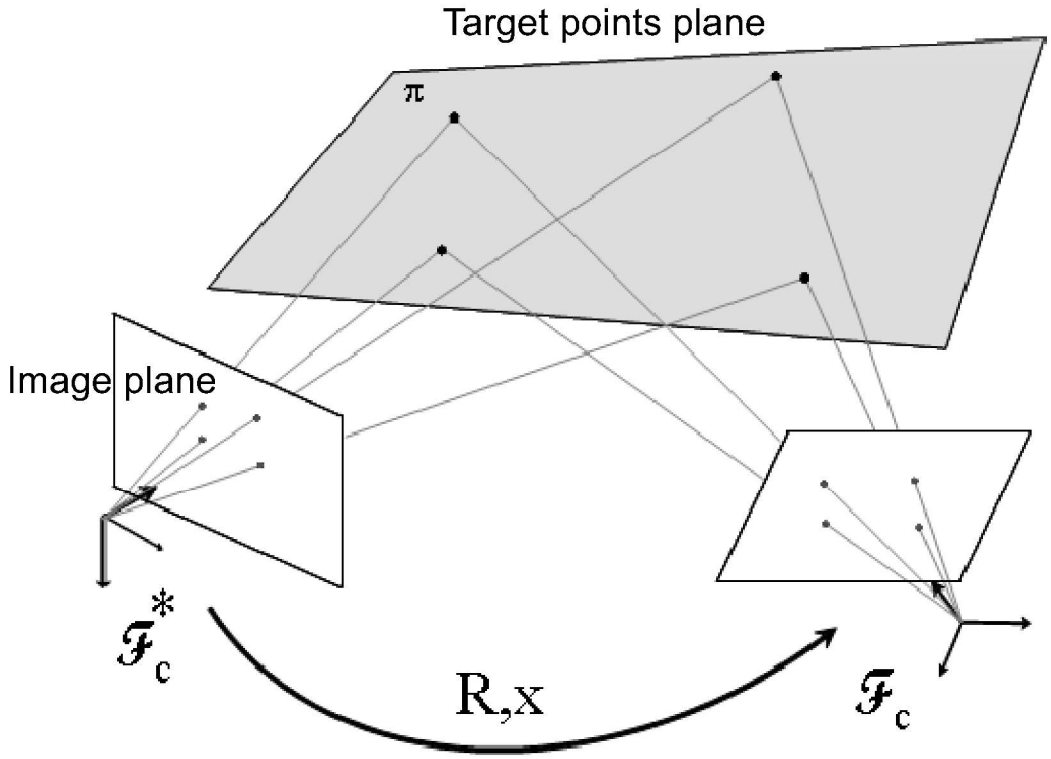
$$m_j^* = [\frac{x_j^*}{z_j^*}, \frac{y_j^*}{z_j^*}, 1]^T$$

Normalized Coordinates

$$p_j^* = Am_j^* = [u^*, v^*, 1]^T$$

$$p_j = Am_j = [u, v, 1]^T \quad A \in \mathbb{R}^{3\times 3}$$

Pixel Coordinates



The image points are formally related by the Rotation R and translation x between camera poses as

$$z_j m_j = z_j^* R m_j^* + x, \quad \forall j \in \{1 \ldots N\}$$

Structure from Motion Review

- Noncoplanar points are related by the Essential Matrix E
- Given eight noncoplanar points we can solve for E, R and λx

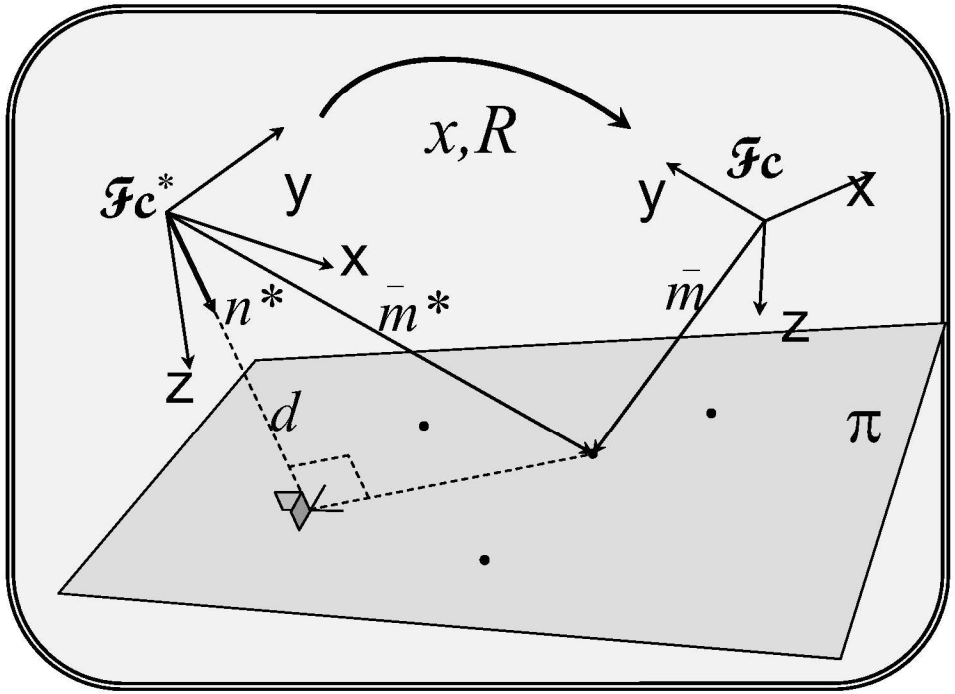
$$m_j^T [x]_\times R m_j^* = 0$$

$$m_j^T E m_j^* = 0$$

- Coplanar points are related by the Euclidean Homography Matrix H and depth ratios α_j
- Given four coplanar points we can solve for H, α_j, R and x/d^*

$$m_j = \frac{z_j^*}{z_j}\left(R + \frac{x}{d^*} n^{*T}\right) m_j^*$$

$$= \alpha_j H m_j^*$$



Camera Kinematics - Background

- Assume camera is fully actuated, 6 DOF, i.e. can move and rotate in any direction
- Rotation matrices are cumbersome in control
  - Not a vector space
  - Not a minimal representation (9 elements to represent 3 angles
- Can locally map rotation matrix to three elements
  - Euler angles
  - Roll, pitch, yaw angles
  - *Angle/axis
  - *Unit quaternions (4 elements)

Camera Kinematics - Background

- Angle/axis representation of Rotation R
  - Rotation of θ about 3D axis u

$$R \rightarrow u\theta \qquad u\theta = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} \theta \qquad u = \frac{1}{2\sin(|\theta|)} \begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix}$$

$$\theta = \text{acos}(\tfrac{1}{2}(Tr(R) - 1))$$

$$u\theta \rightarrow R$$

$$R = e^{[u]_\times \theta} = I_3 + [u]_\times \sin(\theta) + [u]_\times^2 (1 - \cos(\theta))$$

$$[u]_\times \theta = \begin{bmatrix} 0 & -u_3 & u_2 \\ u_3 & 0 & -u_1 \\ -u_2 & u_1 & 0 \end{bmatrix} \theta$$

Camera Kinematics - Background

- Camera pose can then be represented as a 6D vector

$$e_p(t) = [x(t)^T, u(t)^T\theta(t)]^T$$

$x(t) \in \mathbb{R}^3$ translation

$u(t)\theta(t) \in \mathbb{R}^3$ angle/axis rotation

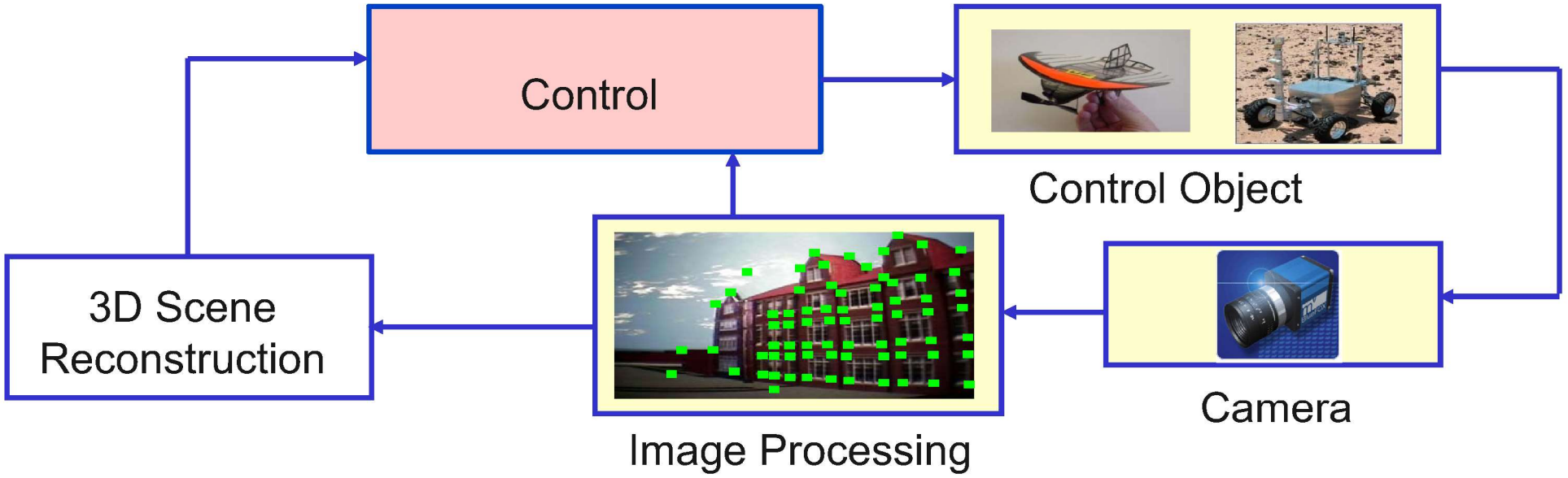
- Camera velocity is also represented as a 6D velocity vector

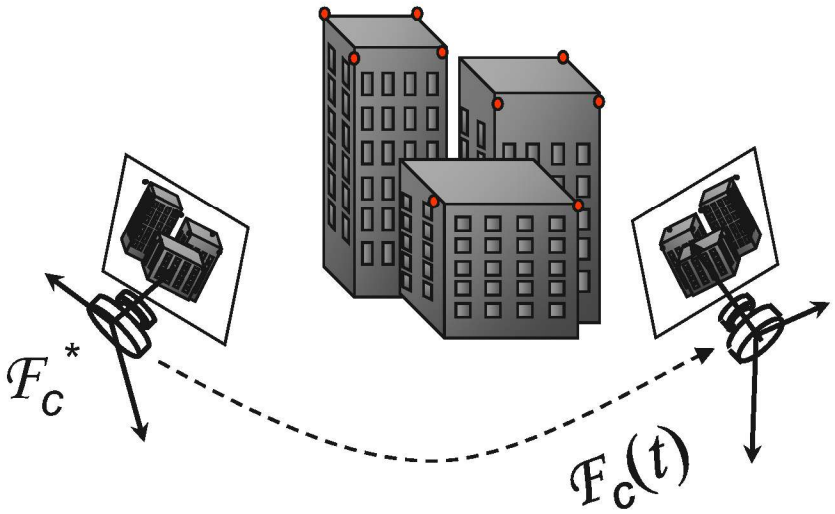
$$\xi(t) = [v(t)^T, \omega^T(t)]^T \in \mathbb{R}^6$$

$v(t) \in \mathbb{R}^3$ linear velocity

$\omega(t) \in \mathbb{R}^3$ angular velocity

Visual Servo Control Problem Description





- A camera is mounted on a control object (e.g. a robot manipulator or vehicle)
- The camera is at a current pose $\mathcal{F}_c(t)$
- There exists a goal pose $\mathcal{F}_c^*$
- The task of visual servoing is to move the camera such that $\mathcal{F}_c(t) \rightarrow \mathcal{F}_c^*$, as $t \rightarrow \infty$
- In some cases $\mathcal{F}_c^*$ is known a priori, in some cases it is unknown, but we have an image taken from $\mathcal{F}_c^*$, (i.e. a goal image)

Position Based Visual Servoing

- Error is defined as a pose in Euclidean space
  - Camera acts as a "Cartesian sensor" to estimate pose error

- Define pose error signal as

$$e_p(t) = [x(t)^T, u(t)^T\theta(t)]^T \qquad x(t) \in \mathbb{R}^3 \qquad u(t)\theta(t) \in \mathbb{R}^3$$

- Open loop error dynamics given as a function of camera velocity

$$\dot{e}_p = L_p\xi \qquad \xi(t) = [v(t)^T, \omega^T(t)]^T \in \mathbb{R}^6$$

Position Based Visual Servoing

$$L_p = \begin{bmatrix} R_{vc} & 0_{3\times3} \\ 0_{3\times3} & R_{vc}L_\omega \end{bmatrix},$$

$$L_\omega = I - \frac{\theta}{2}u_\times + \left(1 - \frac{\mathrm{sinc}(\theta)}{\mathrm{sinc}^2\left(\frac{\theta}{2}\right)}\right)u_\times^2$$

$$\sin c(\theta) = \frac{\sin(\theta)}{\theta}$$

$R_{vc}(t)$ is the rotation matrix from the frame in which ξ(t) is measured to the camera frame.

$R_{vc}(t)$ is an identity matrix if the camera frame and input velocity frame are the same.

Position Based Visual Servoing

- Stabilizing proportional feedback

$$\xi = -k_p L_p^{-1} e_p \qquad k_p \text{ is pos scalar gain}$$

$$L_p^{-1} = \begin{bmatrix} R_{vc}^T & 0_{3\times3} \\ 0_{3\times3} & L_\omega^{-1} R_{vc}^T \end{bmatrix}, L_\omega^{-1} = I + \frac{\theta}{2}\operatorname{sinc}^2\left(\frac{\theta}{2}\right)u_\times + (1 - \operatorname{sinc}(\theta))u_\times^2$$

$$L_\omega^{-1} u\theta = L_\omega u\theta = u\theta$$

- Closed loop error dynamics given by

$$\begin{aligned} \dot{e}_p &= L_p \xi \\ &= L_p(-k_p L_p^{-1} ep) \\ &= -k_p e_p \end{aligned}$$

Position Based Visual Servoing

To prove stability, define Lyapunov function

$$V_p(e_p) = \frac{1}{2} e_p^T e_p,$$

$$= \frac{1}{2} \| e_p(t) \|^2$$

V_p is pos def

With time derivative

$$\dot{V}_p = e_p^T \dot{e}_p$$

$$= e_p^T(-k_p e_p)$$

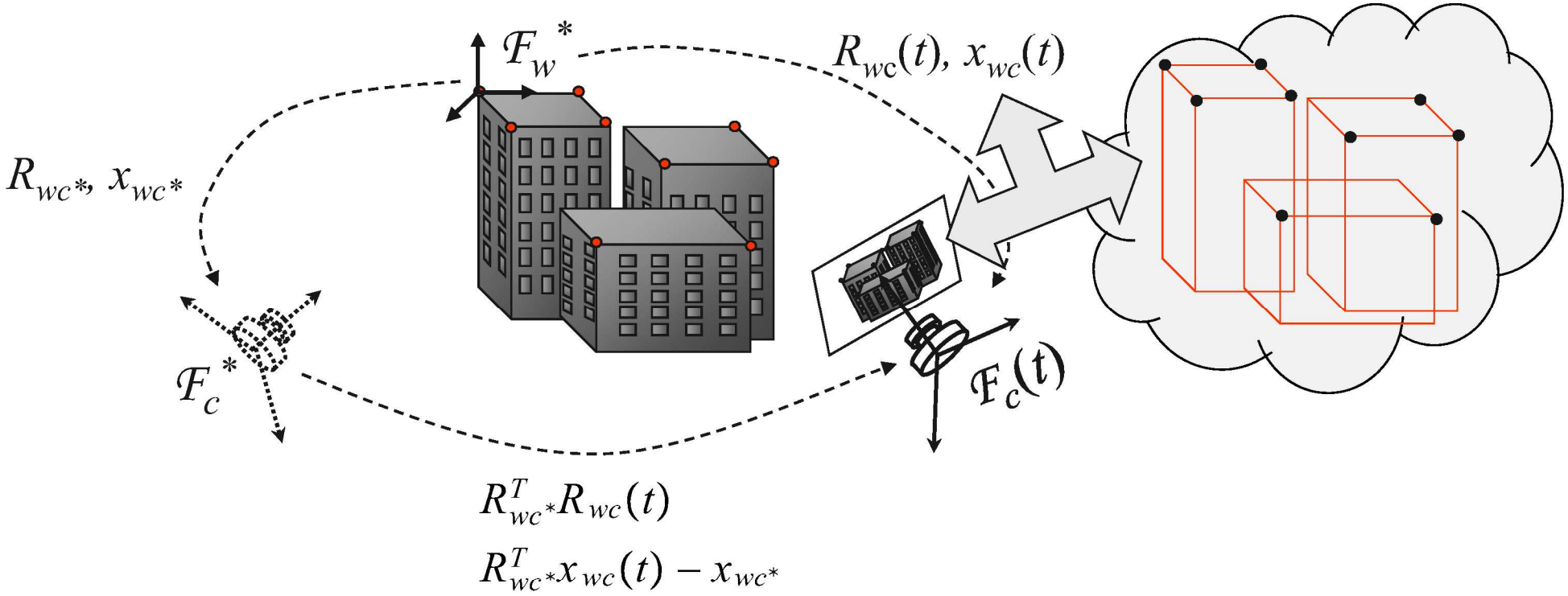
$$= -k_p \| e_p \|^2$$

$\dot{V}_p$ is neg def

Negative definite Lyapunov function means the controller is Globally Asymptotically Stable and $e_p \rightarrow 0$ as $t \rightarrow \infty$

Pose Estimation for PBVS

- Given known desired pose R_{wc*}, x_{wc*}, of camera $\mathcal{F}_c^*$ with respect to $\mathcal{F}_w^*$
- Solve for current pose $R(t)_{wc}$, $x(t)_{wc}$, of camera $\mathcal{F}_c(t)$ with respect to $\mathcal{F}_w^*$
- Solve for current pose $R(t) = R_{wc*}^T R_{wc}(t)$ $\quad x(t) = R_{wc*}^T x_{wc}(t) - x_{wc*}$
  of current camera $\mathcal{F}_c(t)$ with respect to desired pose $\mathcal{F}_c^*$



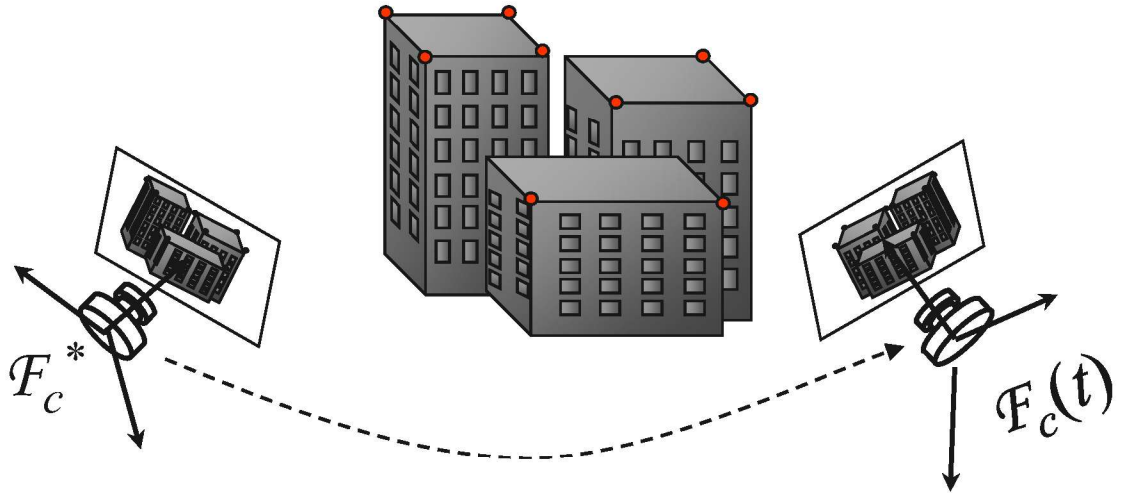
Pose Estimation for PBVS

- If a geometric model is not available, but a goal image is, Homography and Epipolar methods can work

$$m_j^T(t)[x(t)]_\times R(t) m_j^* = 0 \qquad m_j(t) = \frac{z_j^*}{z_j(t)}\left(R(t) + \left(\frac{x(t)}{d^*}\right) n^{*T}\right) m_j^*$$

$$m_j^T(t) E(t) m_j^* = 0 \qquad m_j(t) = \alpha_j(t) H(t) m_j^*$$

- Pose error defined with respect to the pose where a goal image was captured



Pose Estimation for PBVS

$$m_j^T(t)[x(t)]_\times R(t) m_j^* = 0 \qquad m_j(t) = \frac{z_j^*}{z_j(t)}\left(R(t) + \left(\frac{x(t)}{d^*}\right)n^{*T}\right)m_j^*$$

$$m_j^T(t)E(t)m_j^* = 0 \qquad m_j(t) = \alpha_j(t)H(t)m_j^*$$

$$\Updownarrow \qquad\qquad \Updownarrow$$

$$R(t),\ \lambda x(t) \qquad\qquad R(t),\ \frac{x(t)}{d^*}$$

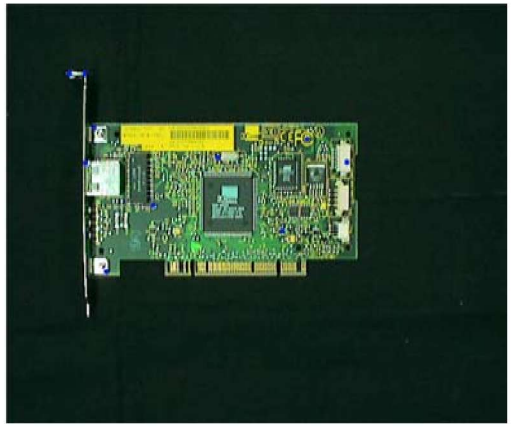
Translation only known up to scale factor, which can cause problems

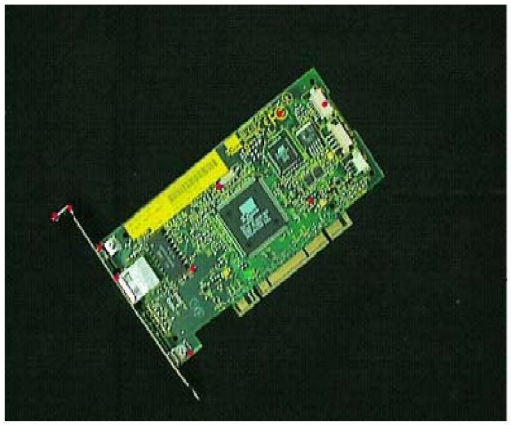
- For essential matrix decomposition, we solve for λx such that $\|\lambda x\|=1$, so velocity can't stabilize once translation error is reduced to $\|x\|<k_p$!
- Even if proper scale is known/measured, E solution breaks down as $x \rightarrow 0$!
- For Homography decomposition, we solve for x/d^*, where d^* is unknown positive constant. Unknown scale might slow down convergence, but does not affect stability.

Method 2: IBVS
Image Based Visual Servoing

Image-Based Visual Servoing

- Error is defined in the image space
  - Measure the difference between the coordinates of several features in the current image and in a goal image
  - The camera is moved such that the features move towards their goal coordinates





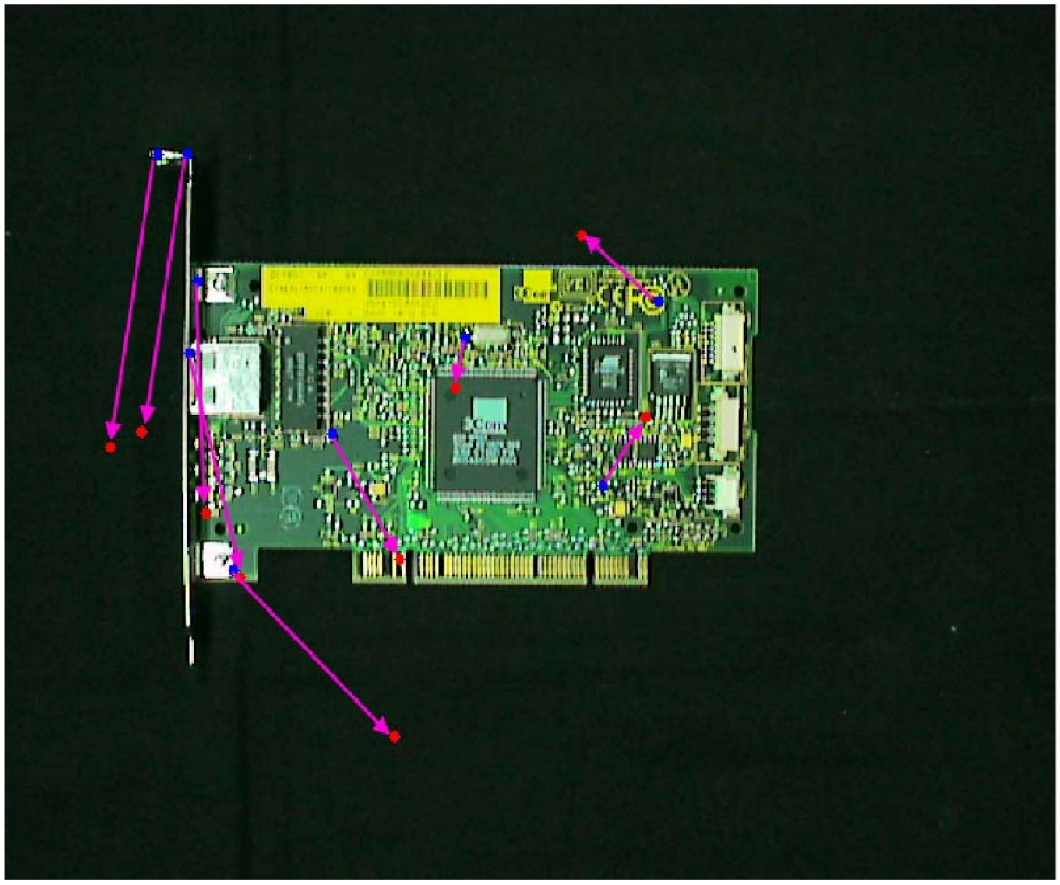


Image Based Visual Servoing

- Rewrite previous equation in matrix form

$$\dot{m}_x = -\frac{1}{Z}v_x + \frac{m_x}{Z}v_z + m_x m_y \omega_x - (1 + m_x^2)\omega_y + m_y \omega_z$$

$$\dot{m}_y = -\frac{1}{Z}v_y + \frac{m_y}{Z}v_z + (1 + m_y^2)\omega_x - m_x m_y \omega_y - m_x \omega_z$$

$$\begin{bmatrix} \dot{m}_x \\ \dot{m}_y \end{bmatrix} = \begin{bmatrix} \frac{1}{Z} & 0 & \frac{m_x}{Z} & m_x m_y & -(1+m_x^2) & m_y \\ 0 & \frac{1}{Z} & \frac{m_y}{Z} & (1+m_y^2) & -m_x m_y & m_x \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ v_z \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

Needs depth estimate

$$\dot{m}_j = L_{ij}\xi$$

- L_{ij} is the Image Jacobian or Interaction Matrix for point j

Image Based Visual Servoing

- Stack point velocities and interaction matrices for all N points

$$\begin{bmatrix} \dot{m}_1 \\ \vdots \\ \dot{m}_N \end{bmatrix} = \begin{bmatrix} L_{i1} \\ \vdots \\ L_{iN} \end{bmatrix} \xi \Rightarrow \dot{m} = L_i \xi$$

- If N=3, L_i is 6x6 and can be inverted if full rank
  - L_i can be singular, depending on coordinates of feature points
- If N>3, L_i is $2N$x6 and we can take pseudo inverse

$$L_i^+ = (L_i^T L_i)^{-1} L_i^T$$

Image Based Visual Servoing

- Error defined in image space as the difference in current feature locations from goal feature locations

$$e_i(t) = \begin{bmatrix} m_1(t) \\ \vdots \\ m_N(t) \end{bmatrix} - \begin{bmatrix} m_1^* \\ \vdots \\ m_N^* \end{bmatrix}$$

- Open loop error dynamics given as a function of camera velocity

$$\dot{e}_i = L_i \xi$$

$$\xi(t) = [v(t)^T, \omega^T(t)]^T \in \mathbb{R}^6$$

Image Based Visual Servoing

To prove stability, define Lyapunov function

$$V_i(t) = \frac{1}{2} e_i^T e_i$$
$$= \frac{1}{2} \| e_i(t) \|^2$$

V_i is pos def

With time derivative

$$\dot{V}_i = e_i^T \dot{e}_i$$
$$= e_i^T (-k_i e_i)$$
$$= -k_i \| e_i \|^2$$

$\dot{V}_i$ is neg def

Negative definite Lyapunov function means the controller is Locally Asymptotically Stable and $e_i \rightarrow 0$ as $t \rightarrow \infty$

PBVS vs IBVS

- PBVS is Globally Asymptotically Stable (GAS) w.r.t. the Cartesian pose error, but no control over the image features
  - GAS is obviously desirable
  - Pose error strictly decreases, the robot follows least distance path to the goal
  - Requires an accurate target model or a goal image for Essential/Homography methods
  - Image features may leave the image
- IBVS is Locally Asymptotically Stable (LAS) w.r.t. the image feature error
  - Neighborhood of AS is unknown, seems large
  - Image error strictly decreases, the feature follow least distance path to their goal coordinates in the image plane
  - Requires a goal image
  - Requires accurate depth estimates for proper performance
  - Pose error can increase without bound